

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет информатика и системы управления
Кафедра системы обработки информации и управления

Курс «Парадигмы и конструкции языков программирования»
Отчет по домашнему заданию

Выполнил:
студент группы ИУ5-32Б:
Кунев В.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Нардид А.Н.
Подпись и дата:

Москва, 2025 г.

Описание задания

Цель работы: Изучение языка программирования C# и платформы .NET 8.0 путем реализации проекта для интеллектуальной синхронизации данных.

Выбранный язык: C# (версия 12).

Инструментарий: Visual Studio Code (с использованием WSL для разработки ядра), Windows App SDK (WinUI 3).

Задача: Разработать приложение «BackupSync» для автоматизированного резервного копирования файлов на внешние USB-носители с учетом их аппаратной идентификации.

Описание проекта и изученных аспектов языка

Архитектура решения

Проект разделен на два основных слоя для обеспечения чистоты кода по паттерну MVVM:

1. **BackupSync.Core**: Библиотека классов, содержащая бизнес-логику, модели данных и взаимодействие с БД.
2. **BackupSync.UI**: Слой представления на базе WinUI 3 для создания современного интерфейса в стиле Fluent Design.

Основные возможности и изученные конструкции

В процессе разработки были изучены и применены следующие особенности C#:

- **Асинхронное программирование (async/await)**: Использовано для неблокирующего выполнения файловых операций и запросов к БД.
- **LINQ (Language Integrated Query)**: Применяется для фильтрации и обработки коллекций данных, например, при сопоставлении физических дисков с профилями в БД.
- **Entity Framework Core**: Изучено объектно-реляционное отображение (ORM) для работы с SQLite.
- **WMI (Windows Management Instrumentation)**: Реализован низкоуровневый мониторинг системы для обнаружения подключения USB-устройств.

Текст программы

```
=== BackupSync.Core\Abstractions\IFileSystemProvider.cs ===
using System;
using System.Collections.Generic;
using System.Threading.Tasks;

namespace BackupSync.Core.Abstractions
{
    /// <summary>
    /// Модель, содержащая основные метаданные файла, необходимые для сравнения.
    /// Используется для минимизации операций ввода-вывода (не читаем контент
    файла).
    /// </summary>
    public class FileMetadata
    {
        /// <summary>
        /// Относительный путь к файлу (от корня сканирования).
        /// </summary>
        public string RelativePath { get; set; } = string.Empty;

        /// <summary>
        /// Размер файла в байтах.
        /// </summary>
        public long Size { get; set; }

        /// <summary>
        /// Время последнего изменения файла (в UTC).
        /// </summary>
        public DateTime LastWriteTimeUtc { get; set; }
    }

    /// <summary>
    /// Абстракция для операций с файловой системой.
    /// Позволяет абстрагироваться от конкретного источника (локальный диск,
    сетевая папка, VSS-снимок).
    /// </summary>
    public interface IFileSystemProvider
    {
        /// <summary>
        /// Асинхронно сканирует указанную директорию и возвращает словарь
        метаданных всех найденных файлов.
        /// </summary>
        /// <param name="rootPath">Корневой путь для сканирования
        (абсолютный).</param>
        /// <param name="exclusions">Список шаблонов имен файлов/папок для
        исключения из сканирования.</param>
        /// <returns>
        /// Словарь, где:
        /// Key - относительный путь к файлу,
        /// Value - объект <see cref="FileMetadata"/>.
    }
}
```

```

        /// </returns>
        /// <remarks>
        /// TODO [Future]: Реализовать логику фильтрации на основе параметра
        exclusions.
        /// TODO [VSS]: При работе с VSS здесь будет передаваться путь к
        смонтированному снимку.
        /// </remarks>
        Task<Dictionary<string, FileMetadata>> ScanDirectoryAsync(string
        rootPath, List<string> exclusions);

        /// <summary>
        /// Асинхронно копирует один файл из источника в назначение.
        /// Если файл назначения существует, он должен быть перезаписан.
        /// </summary>
        /// <param name="sourceFullPath">Полный путь к файлу-источнику.</param>
        /// <param name="destFullPath">Полный путь к файлу-назначения.</param>
        Task CopyFileAsync(string sourceFullPath, string destFullPath);

        /// <summary>
        /// Асинхронно удаляет файл по указанному пути.
        /// Если файл не существует, метод должен завершаться успешно
        (идемпотентность).
        /// </summary>
        /// <param name="fullPath">Полный путь к удаляемому файлу.</param>
        Task DeleteFileAsync(string fullPath);
    }
}

```

```

=== BackupSync.Core\Migrations\20251222205251_InitialCreate.cs ===
using Microsoft.EntityFrameworkCore.Migrations;

```

```

#nullable disable

```

```

namespace BackupSync.Core.Migrations
{
    /// <inheritdoc />
    public partial class InitialCreate : Migration
    {
        /// <inheritdoc />
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "Devices",
                columns: table => new
                {
                    DeviceId = table.Column<int>(type: "INTEGER", nullable:
false)
                        .Annotation("Sqlite:Autoincrement", true),
                    UserGivenName = table.Column<string>(type: "TEXT", maxLength:
100, nullable: false),

```

```

        VolumeSerialNumber = table.Column<string>(type: "TEXT",
maxLength: 50, nullable: false),
        LastKnownLabel = table.Column<string>(type: "TEXT",
maxLength: 100, nullable: true),
        LastKnownLetter = table.Column<string>(type: "TEXT",
maxLength: 10, nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_Devices", x => x.DeviceId);
    });

migrationBuilder.CreateTable(
    name: "SyncJobs",
    columns: table => new
    {
        SyncJobId = table.Column<int>(type: "INTEGER", nullable:
false)
            .Annotation("Sqlite:Autoincrement", true),
        JobName = table.Column<string>(type: "TEXT", maxLength: 100,
nullable: false),
        SourcePath = table.Column<string>(type: "TEXT", maxLength:
1024, nullable: false),
        DestPath = table.Column<string>(type: "TEXT", maxLength:
1024, nullable: false),
        DeviceId = table.Column<int>(type: "INTEGER", nullable:
false),
        JobType = table.Column<int>(type: "INTEGER", nullable:
false),
        OrphanPolicy = table.Column<int>(type: "INTEGER", nullable:
false),
        ConflictPolicy = table.Column<int>(type: "INTEGER", nullable:
false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_SyncJobs", x => x.SyncJobId);
        table.ForeignKey(
            name: "FK_SyncJobs_Devices_DeviceId",
            column: x => x.DeviceId,
            principalTable: "Devices",
            principalColumn: "DeviceId",
            onDelete: ReferentialAction.Cascade);
    });

migrationBuilder.CreateTable(
    name: "JobExclusions",
    columns: table => new
    {
        JobExclusionId = table.Column<int>(type: "INTEGER", nullable:
false)

```

```

        .Annotation("Sqlite:Autoincrement", true),
        Pattern = table.Column<string>(type: "TEXT", maxLength: 255,
nullable: false),
        SyncJobId = table.Column<int>(type: "INTEGER", nullable:
false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_JobExclusions", x => x.JobExclusionId);
        table.ForeignKey(
            name: "FK_JobExclusions_SyncJobs_SyncJobId",
            column: x => x.SyncJobId,
            principalTable: "SyncJobs",
            principalColumn: "SyncJobId",
            onDelete: ReferentialAction.Cascade);
    });

    migrationBuilder.CreateIndex(
        name: "IX_JobExclusions_SyncJobId",
        table: "JobExclusions",
        column: "SyncJobId");

    migrationBuilder.CreateIndex(
        name: "IX_SyncJobs_DeviceId",
        table: "SyncJobs",
        column: "DeviceId");
}

/// <inheritdoc />
protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropTable(
        name: "JobExclusions");

    migrationBuilder.DropTable(
        name: "SyncJobs");

    migrationBuilder.DropTable(
        name: "Devices");
}
}
}

```

```

=== BackupSync.Core\Migrations\20251222205251_InitialCreate.Designer.cs ===
// <auto-generated />
using BackupSync.Core.Services;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Infrastructure;
using Microsoft.EntityFrameworkCore.Migrations;
using Microsoft.EntityFrameworkCore.Storage.ValueConversion;

```

```
#nullable disable
```

```
namespace BackupSync.Core.Migrations
```

```
{
    [DbContext(typeof(DatabaseContext))]
    [Migration("2025122205251_InitialCreate")]
    partial class InitialCreate
    {
        /// <inheritdoc />
        protected override void BuildTargetModel(ModelBuilder modelBuilder)
        {
#pragma warning disable 612, 618
            modelBuilder.HasAnnotation("ProductVersion", "9.0.10");

            modelBuilder.Entity("BackupSync.Core.Models.Device", b =>
            {
                b.Property<int>("DeviceId")
                    .ValueGeneratedOnAdd()
                    .HasColumnType("INTEGER");

                b.Property<string>("LastKnownLabel")
                    .HasMaxLength(100)
                    .HasColumnType("TEXT");

                b.Property<string>("LastKnownLetter")
                    .HasMaxLength(10)
                    .HasColumnType("TEXT");

                b.Property<string>("UserGivenName")
                    .IsRequired()
                    .HasMaxLength(100)
                    .HasColumnType("TEXT");

                b.Property<string>("VolumeSerialNumber")
                    .IsRequired()
                    .HasMaxLength(50)
                    .HasColumnType("TEXT");

                b.HasKey("DeviceId");

                b.ToTable("Devices");
            });

            modelBuilder.Entity("BackupSync.Core.Models.JobExclusion", b =>
            {
                b.Property<int>("JobExclusionId")
                    .ValueGeneratedOnAdd()
                    .HasColumnType("INTEGER");

                b.Property<string>("Pattern")
```



```

        .IsRequired()
        .HasMaxLength(255)
        .HasColumnType("TEXT");

b.Property<int>("SyncJobId")
    .HasColumnType("INTEGER");

b.HasKey("JobExclusionId");

b.HasIndex("SyncJobId");

b.ToTable("JobExclusions");
});

modelBuilder.Entity("BackupSync.Core.Models.SyncJob", b =>
{
    b.Property<int>("SyncJobId")
        .ValueGeneratedOnAdd()
        .HasColumnType("INTEGER");

    b.Property<int>("ConflictPolicy")
        .HasColumnType("INTEGER");

    b.Property<string>("DestPath")
        .IsRequired()
        .HasMaxLength(1024)
        .HasColumnType("TEXT");

    b.Property<int>("DeviceId")
        .HasColumnType("INTEGER");

    b.Property<string>("JobName")
        .IsRequired()
        .HasMaxLength(100)
        .HasColumnType("TEXT");

    b.Property<int>("JobType")
        .HasColumnType("INTEGER");

    b.Property<int>("OrphanPolicy")
        .HasColumnType("INTEGER");

    b.Property<string>("SourcePath")
        .IsRequired()
        .HasMaxLength(1024)
        .HasColumnType("TEXT");

    b.HasKey("SyncJobId");

    b.HasIndex("DeviceId");

```

```

        b.ToTable("SyncJobs");
    });

    modelBuilder.Entity("BackupSync.Core.Models.JobExclusion", b =>
    {
        b.HasOne("BackupSync.Core.Models.SyncJob", "SyncJob")
            .WithMany("Exclusions")
            .HasForeignKey("SyncJobId")
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();

        b.Navigation("SyncJob");
    });

    modelBuilder.Entity("BackupSync.Core.Models.SyncJob", b =>
    {
        b.HasOne("BackupSync.Core.Models.Device", "Device")
            .WithMany("SyncJobs")
            .HasForeignKey("DeviceId")
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();

        b.Navigation("Device");
    });

    modelBuilder.Entity("BackupSync.Core.Models.Device", b =>
    {
        b.Navigation("SyncJobs");
    });

    modelBuilder.Entity("BackupSync.Core.Models.SyncJob", b =>
    {
        b.Navigation("Exclusions");
    });
#pragma warning restore 612, 618
    }
}
}

```

```

=== BackupSync.Core\Migrations\DatabaseContextModelSnapshot.cs ===
// <auto-generated />
using BackupSync.Core.Services;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Infrastructure;
using Microsoft.EntityFrameworkCore.Storage.ValueConversion;

#nullable disable

namespace BackupSync.Core.Migrations
{

```

```

[DbContext(typeof(DatabaseContext))]
partial class DatabaseContextModelSnapshot : ModelSnapshot
{
    protected override void BuildModel(ModelBuilder modelBuilder)
    {
#pragma warning disable 612, 618
        modelBuilder.HasAnnotation("ProductVersion", "9.0.10");

        modelBuilder.Entity("BackupSync.Core.Models.Device", b =>
        {
            b.Property<int>("DeviceId")
                .ValueGeneratedOnAdd()
                .HasColumnType("INTEGER");

            b.Property<string>("LastKnownLabel")
                .HasMaxLength(100)
                .HasColumnType("TEXT");

            b.Property<string>("LastKnownLetter")
                .HasMaxLength(10)
                .HasColumnType("TEXT");

            b.Property<string>("UserGivenName")
                .IsRequired()
                .HasMaxLength(100)
                .HasColumnType("TEXT");

            b.Property<string>("VolumeSerialNumber")
                .IsRequired()
                .HasMaxLength(50)
                .HasColumnType("TEXT");

            b.HasKey("DeviceId");

            b.ToTable("Devices");
        });

        modelBuilder.Entity("BackupSync.Core.Models.JobExclusion", b =>
        {
            b.Property<int>("JobExclusionId")
                .ValueGeneratedOnAdd()
                .HasColumnType("INTEGER");

            b.Property<string>("Pattern")
                .IsRequired()
                .HasMaxLength(255)
                .HasColumnType("TEXT");

            b.Property<int>("SyncJobId")
                .HasColumnType("INTEGER");

```

```

        b.HasKey("JobExclusionId");

        b.HasIndex("SyncJobId");

        b.ToTable("JobExclusions");
    });

modelBuilder.Entity("BackupSync.Core.Models.SyncJob", b =>
{
    b.Property<int>("SyncJobId")
        .ValueGeneratedOnAdd()
        .HasColumnType("INTEGER");

    b.Property<int>("ConflictPolicy")
        .HasColumnType("INTEGER");

    b.Property<string>("DestPath")
        .IsRequired()
        .HasMaxLength(1024)
        .HasColumnType("TEXT");

    b.Property<int>("DeviceId")
        .HasColumnType("INTEGER");

    b.Property<string>("JobName")
        .IsRequired()
        .HasMaxLength(100)
        .HasColumnType("TEXT");

    b.Property<int>("JobType")
        .HasColumnType("INTEGER");

    b.Property<int>("OrphanPolicy")
        .HasColumnType("INTEGER");

    b.Property<string>("SourcePath")
        .IsRequired()
        .HasMaxLength(1024)
        .HasColumnType("TEXT");

    b.HasKey("SyncJobId");

    b.HasIndex("DeviceId");

    b.ToTable("SyncJobs");
});

modelBuilder.Entity("BackupSync.Core.Models.JobExclusion", b =>
{
    b.HasOne("BackupSync.Core.Models.SyncJob", "SyncJob")
        .WithMany("Exclusions")

```

```

        .HasForeignKey("SyncJobId")
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();

        b.Navigation("SyncJob");
    });

modelBuilder.Entity("BackupSync.Core.Models.SyncJob", b =>
{
    b.HasOne("BackupSync.Core.Models.Device", "Device")
        .WithMany("SyncJobs")
        .HasForeignKey("DeviceId")
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();

    b.Navigation("Device");
});

modelBuilder.Entity("BackupSync.Core.Models.Device", b =>
{
    b.Navigation("SyncJobs");
});

modelBuilder.Entity("BackupSync.Core.Models.SyncJob", b =>
{
    b.Navigation("Exclusions");
});
#pragma warning restore 612, 618
}
}
}

```

```

=== BackupSync.Core\Models\ConflictPolicy.cs ===
namespace BackupSync.Core.Models
{
    /// <summary>
    /// Политика разрешения конфликтов синхронизации.
    /// Определяет действие, если файл существует и в Источнике (Source), и в
    Назначении (Dest), но их метаданные отличаются.
    /// </summary>
    public enum ConflictPolicy
    {
        /// <summary>
        /// Спрашивать пользователя о действии для каждого конфликта.
        /// </summary>
        Ask = 0,

        /// <summary>
        /// Всегда оставлять версию с компьютера (Источник/Source).
        /// Файл на носителе будет перезаписан локальной версией.
    }
}

```

```

    /// </summary>
    KeepLocal = 1,

    /// <summary>
    /// Всегда оставлять версию с носителя (Назначение/Dest).
    /// Локальный файл будет перезаписан версией с носителя (если это
    подразумевает тип синхронизации).
    /// </summary>
    KeepRemote = 2,

    /// <summary>
    /// Автоматически выбирать файл с более поздней датой изменения
    (LastWriteTimeUtc).
    /// </summary>
    KeepNewest = 3
}
}

```

```

=== BackupSync.Core\Models\Device.cs ===
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

namespace BackupSync.Core.Models
{
    /// <summary>
    /// Сущность, представляющая физический внешний носитель (USB-флешку, HDD),
    сохраненный в базе данных.
    /// Используется для привязки задач синхронизации к конкретному "железу".
    /// </summary>
    public class Device
    {
        /// <summary>
        /// Уникальный идентификатор устройства в БД (Primary Key).
        /// </summary>
        [Key]
        public int DeviceId { get; set; }

        /// <summary>
        /// Пользовательское имя устройства (например, "Рабочая флешка").
        /// </summary>
        [Required]
        [MaxLength(100)]
        public string UserGivenName { get; set; } = string.Empty;

        /// <summary>
        /// Серийный номер тома (Volume Serial Number), получаемый через WMI
        (например, "E4B0-1A2C").
        /// Является основным уникальным идентификатором физического носителя.
        /// </summary>
        [Required]

```

```

[MaxLength(50)]
public string VolumeSerialNumber { get; set; } = string.Empty;

/// <summary>
/// Последняя известная метка тома (например, "MY_BACKUP").
/// Используется для UI, когда устройство отключено.
/// </summary>
[MaxLength(100)]
public string? LastKnownLabel { get; set; }

/// <summary>
/// Последняя известная буква диска (например, "E:").
/// Примечание: Буква может меняться при переподключении.
/// </summary>
[MaxLength(10)]
public string? LastKnownLetter { get; set; }

/// <summary>
/// Навигационное свойство: Список задач синхронизации, привязанных к
этому устройству.
/// </summary>
public List<SyncJob> SyncJobs { get; set; } = new List<SyncJob>();
}
}

```

=== BackupSync.Core\Models\DeviceStatus.cs ===

```

using System;

namespace BackupSync.Core.Models
{
    /// <summary>
    /// Модель состояния устройства для отображения в UI.
    /// Объединяет данные из базы данных (сохраненные настройки) и WMI (текущее
    физическое состояние).
    /// </summary>
    public class DeviceStatus
    {
        // --- Данные из WMI (Физическое подключение) ---

        /// <summary>
        /// Объект физического диска (если подключен). Null, если устройство
        отключено.
        /// </summary>
        public LogicalDisk? WmiDisk { get; set; }

        // --- Данные из БД (Сохраненный профиль) ---

        /// <summary>
        /// ID устройства в базе данных (если опознано).
        /// </summary>

```

```

public int? DeviceId { get; set; }

/// <summary>
/// Имя, данное пользователем (если опознано).
/// </summary>
public string? UserGivenName { get; set; }

// --- Вычисляемые статусы ---

/// <summary>
/// Подключено ли устройство физически прямо сейчас.
/// </summary>
public bool IsConnected { get; set; }

/// <summary>
/// Есть ли это устройство в нашей базе данных.
/// </summary>
public bool IsRecognized { get; set; }

// --- Умные свойства для UI (Binding) ---

/// <summary>
/// Форматированное имя для отображения в списке.
/// Логика: "Имя пользователя (Метка тома)" или просто "Метка тома".
/// </summary>
public string DisplayName
{
    get
    {
        // 1. Получаем системную метку (например, "KINGSTON")
        string? systemLabel = WmiDisk?.VolumeLabel;
        bool hasSystemLabel = !string.IsNullOrEmpty(systemLabel);

        // 2. Если устройство ОПОЗНАНО (есть в БД)
        if (IsRecognized)
        {
            string userName = UserGivenName ?? "Без имени";

            // Если есть системная метка и она отличается от имени
            // пользователя
            // (например: Юзер назвал "Работа", а флешка называется
            // "USB_DISK")
            if (hasSystemLabel && !string.Equals(userName, systemLabel,
StringComparison.OrdinalIgnoreCase))
            {
                return $"{userName} ({systemLabel})";
            }

            // Если метки совпадают или системной нет -> показываем
            // только имя пользователя
            return userName;
        }
    }
}

```



```

    }

    // 3. Если устройство НЕ ОПОЗНАНО (нет в БД)
    // Показываем метку тома или заглушку
    return hasSystemLabel ? systemLabel! : "Съемный носитель";
}
}

/// <summary>
/// Буква диска (например, "E:"). Возвращает "-", если диск отключен.
/// </summary>
public string Letter => WmiDisk?.Letter ?? "-";

/// <summary>
/// Серийный номер тома. Возвращает "-", если диск отключен.
/// </summary>
public string VolumeSerialNumber => WmiDisk?.VolumeSerialNumber ?? "-";

/// <summary>
/// Флаг для UI: можно ли перепривязать это устройство (оно в БД, но
отключено).
/// </summary>
public bool CanBeReAssociated => !IsConnected && IsRecognized;
}
}

```

```

=== BackupSync.Core\Models\JobExclusion.cs ===
using System.ComponentModel.DataAnnotations;

namespace BackupSync.Core.Models
{
    /// <summary>
    /// Сущность, представляющая правило исключения файлов или папок из процесса
    синхронизации.
    /// Позволяет игнорировать временные файлы, системные директории и т.д.
    /// </summary>
    public class JobExclusion
    {
        /// <summary>
        /// Уникальный идентификатор правила (Primary Key).
        /// </summary>
        [Key]
        public int JobExclusionId { get; set; }

        /// <summary>
        /// Шаблон исключения. Поддерживает простые маски.
        /// Примеры: "*.tmp", "node_modules/", ".git", "Thumbs.db".
        /// </summary>
        [Required]
        [MaxLength(255)]
    }
}

```

```

        public string Pattern { get; set; } = string.Empty;

        // --- Связи (Relationships) ---

        /// <summary>
        /// Внешний ключ (Foreign Key) на задание синхронизации.
        /// </summary>
        public int SyncJobId { get; set; }

        /// <summary>
        /// Навигационное свойство: Задание, к которому относится это правило.
        /// </summary>
        public SyncJob SyncJob { get; set; } = null!;
    }
}

```

```

        === BackupSync.Core\Models\JobType.cs ===
namespace BackupSync.Core.Models
{
    /// <summary>
    /// Тип задания (режим синхронизации).
    /// Определяет направление передачи данных между Источником (PC) и
    Назначением (USB).
    /// </summary>
    public enum JobType
    {
        /// <summary>
        /// Одностороннее резервное копирование (Backup).
        /// Данные копируются строго из Источника в Назначение.
        /// Изменения на носителе не влияют на файлы на компьютере.
        /// </summary>
        OneWay = 1,

        /// <summary>
        /// Двусторонняя синхронизация (Sync).
        /// Данные синхронизируются в обоих направлениях.
        /// Актуальной считается версия с более поздней датой изменения.
        /// </summary>
        TwoWay = 2
    }
}

```

```

        === BackupSync.Core\Models\LogicalDisk.cs ===
namespace BackupSync.Core.Models
{
    /// <summary>
    /// Модель данных (DTO), представляющая логический диск, обнаруженный в
    системе через WMI.

```

```

    /// Используется для передачи информации о физически подключенных
    устройствах.
    /// </summary>
    public class LogicalDisk
    {
        /// <summary>
        /// Буква диска (свойство DeviceID из WMI).
        /// Пример: "E:"
        /// </summary>
        public string Letter { get; set; } = string.Empty;

        /// <summary>
        /// Метка тома (название диска). Может быть null, если метка не задана.
        /// Пример: "MY_BACKUP"
        /// </summary>
        public string? VolumeLabel { get; set; }

        /// <summary>
        /// Серийный номер тома (Volume Serial Number).
        /// Генерируется при форматировании. Используется нами как уникальный
        идентификатор устройства.
        /// Пример: "E4B0-1A2C"
        /// </summary>
        public string? VolumeSerialNumber { get; set; }
    }
}

```

```

    === BackupSync.Core\Models\OrphanPolicy.cs ===
namespace BackupSync.Core.Models
{
    /// <summary>
    /// Политика обработки "сирот".
    /// "Сирота" – это файл или папка, которая существует в Назначении
    (Dest/USB), но отсутствует в Источнике (Source/PC).
    /// Обычно это происходит, если файл был удален на компьютере.
    /// </summary>
    public enum OrphanPolicy
    {
        /// <summary>
        /// Спрашивать пользователя о действии для каждого найденного "сироты".
        /// </summary>
        Ask = 0,

        /// <summary>
        /// Ничего не делать (оставить файл на носителе).
        /// Файл будет проигнорирован и останется на диске, даже если его нет на
        компьютере.
        /// </summary>
        KeepOnRemote = 1,
    }
}

```

```

        /// <summary>
        /// Автоматически удалить файл с носителя.
        /// Используется для режима "Зеркало" (Mirroring): если файл удален на
ПК, он удаляется и на флешке.
        /// </summary>
        AutoDeleteOnRemote = 2,

        /// <summary>
        /// Автоматически скопировать файл обратно на компьютер.
        /// Используется в Двусторонней синхронизации: файл считается "новым",
созданным на другом устройстве.
        /// </summary>
        AutoCopyToLocal = 3
    }
}

```

```

=== BackupSync.Core\Models\SyncAnalysis.cs ===
using BackupSync.Core.Abstractions;

namespace BackupSync.Core.Models
{
    // В этом файле собраны типы, описывающие результаты сравнения файлов и
принятые решения.

    /// <summary>
    /// Результат сравнения двух файлов (или отсутствия одного из них) на этапе
Анализа.
    /// Определяет фактическое состояние файловой системы без привязки к тому,
что с этим делать.
    /// </summary>
    public enum FileStatus
    {
        /// <summary>
        /// Файл существует только в источнике (Source).
        /// Обычно требует копирования в назначение.
        /// </summary>
        OnlyInSource,

        /// <summary>
        /// Файл существует только в назначении (Dest).
        /// Называется "Сирота" (Orphan). Может быть удален или оставлен, в
зависимости от политики.
        /// </summary>
        OnlyInDest,

        /// <summary>
        /// Файлы существуют в обоих местах и считаются идентичными (совпадают
размер и время изменения).
        /// </summary>
        Identical,
    }
}

```

```

    /// <summary>
    /// Файлы существуют в обоих местах, но версия в источнике (Source)
новее.
    /// </summary>
    SourceIsNewer,

    /// <summary>
    /// Файлы существуют в обоих местах, но версия в назначении (Dest) новее.
    /// В режиме One-Way Backup это считается конфликтом. В Two-Way Sync это
нормальная ситуация.
    /// </summary>
    DestIsNewer,
}

/// <summary>
/// Объект, описывающий конкретное различие между Источником и Назначением
для одного пути.
/// Содержит метаданные обеих сторон (если они есть).
/// </summary>
public class FileDiff
{
    /// <summary>
    /// Относительный путь к файлу (например, "Documents\Report.docx").
    /// </summary>
    public string RelativePath { get; set; } = string.Empty;

    /// <summary>
    /// Статус сравнения (Результат анализа).
    /// </summary>
    public FileStatus Status { get; set; }

    /// <summary>
    /// Метаданные файла из Источника (Source). Null, если файл отсутствует в
источнике.
    /// </summary>
    public FileMetadata? SourceMetadata { get; set; }

    /// <summary>
    /// Метаданные файла из Назначения (Dest). Null, если файл отсутствует в
назначении.
    /// </summary>
    public FileMetadata? DestMetadata { get; set; }
}

/// <summary>
/// Тип действия, которое необходимо выполнить с файлом (Результат применения
Политик).
/// </summary>
public enum SyncActionType
{

```

```

    /// <summary>
    /// Скопировать файл из Источника в Назначение (создать новый или
перезаписать старый).
    /// </summary>
    CopyFromSourceToDest,

    /// <summary>
    /// Удалить файл из Назначения (Dest).
    /// Применяется для очистки "сирот" в режиме зеркалирования.
    /// </summary>
    DeleteOnDest,

    /// <summary>
    /// Скопировать файл из Назначения в Источник.
    /// Используется только в режиме Двусторонней синхронизации (Two-Way).
    /// </summary>
    CopyFromDestToSource,

    /// <summary>
    /// Удалить файл из Источника.
    /// Опасно. Используется редко, только при полной синхронизации удалений
в Two-Way режиме.
    /// </summary>
    DeleteOnSource,

    /// <summary>
    /// Пропустить файл (ничего не делать).
    /// </summary>
    Skip,

    /// <summary>
    /// Автоматическое решение невозможно. Требуется вмешательство
пользователя.
    /// </summary>
    AskUser
}

    /// <summary>
    /// Конкретная инструкция к выполнению. Связывает найденное различие (Diff) с
решением (ActionType).
    /// </summary>
    public class SyncAction
    {
        /// <summary>
        /// Тип действия, которое нужно выполнить.
        /// </summary>
        public SyncActionType ActionType { get; set; }

        /// <summary>
        /// Информация о файле и его статусе.
        /// </summary>

```

```

        public FileDiff Diff { get; set; } = null!;
    }
}

```

=== BackupSync.Core\Models\SyncJob.cs ===

```

using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

namespace BackupSync.Core.Models
{
    /// <summary>
    /// Сущность, представляющая одно задание синхронизации (профиль).
    /// Содержит информацию о том, ЧТО, ОТКУДА, КУДА и КАК копировать.
    /// </summary>
    public class SyncJob
    {
        /// <summary>
        /// Уникальный идентификатор задания (Primary Key).
        /// </summary>
        [Key]
        public int SyncJobId { get; set; }

        /// <summary>
        /// Название задания для отображения в списке.
        /// Пример: "Бэкап фото", "Синхронизация работы".
        /// </summary>
        [Required]
        [MaxLength(100)]
        public string JobName { get; set; } = string.Empty;

        /// <summary>
        /// Абсолютный путь к исходной папке на компьютере.
        /// Пример: "D:\Projects\MyProject"
        /// </summary>
        [Required]
        [MaxLength(1024)] // Разумный лимит для путей
        public string SourcePath { get; set; } = string.Empty;

        /// <summary>
        /// Относительный путь к папке назначения на внешнем носителе (от корня
        диска).
        /// Не включает букву диска, так как она может меняться.
        /// Пример: "Backups\MyProject" (в итоге станет "E:\Backups\MyProject").
        /// </summary>
        [Required]
        [MaxLength(1024)]
        public string DestPath { get; set; } = string.Empty;

        // --- Связи (Relationships) ---
    }
}

```

```

        /// <summary>
        /// Внешний ключ (Foreign Key) на устройство, к которому привязано
задание.
        /// </summary>
        public int DeviceId { get; set; }

        /// <summary>
        /// Навигационное свойство: Устройство (флешка/диск), с которым работает
это задание.
        /// </summary>
        public Device Device { get; set; } = null!;

        // --- Политики (Rules) ---

        /// <summary>
        /// Тип синхронизации (Односторонняя / Двусторонняя).
        /// </summary>
        [Required]
        public JobType JobType { get; set; }

        /// <summary>
        /// Политика обработки файлов, которые есть в Назначении, но нет в
Источнике.
        /// </summary>
        [Required]
        public OrphanPolicy OrphanPolicy { get; set; }

        /// <summary>
        /// Политика разрешения конфликтов (когда файл изменен с обеих сторон).
        /// </summary>
        [Required]
        public ConflictPolicy ConflictPolicy { get; set; }

        /// <summary>
        /// Навигационное свойство: Список правил исключения файлов (фильтры).
        /// </summary>
        public List<JobExclusion> Exclusions { get; set; } = new
List<JobExclusion>();
    }
}

```

```

=== BackupSync.Core\Services\DatabaseContext.cs ===
using System;
using System.IO;
using BackupSync.Core.Models;
using Microsoft.EntityFrameworkCore;

namespace BackupSync.Core.Services
{
    /// <summary>

```



```

    /// Контекст базы данных приложения (SQLite).
    /// Отвечает за соединение с БД, маппинг объектов на таблицы и выполнение
запросов.
    /// </summary>
    public class DatabaseContext : DbContext
    {
        /// --- Таблицы (DbSets) ---
        /// Инициализируем как null!, так как EF Core заполняет их через
рефлексию.

        /// <summary>
        /// Таблица устройств (физических носителей).
        /// </summary>
        public DbSet<Device> Devices { get; set; } = null!;

        /// <summary>
        /// Таблица заданий синхронизации.
        /// </summary>
        public DbSet<SyncJob> SyncJobs { get; set; } = null!;

        /// <summary>
        /// Таблица правил исключений для заданий.
        /// </summary>
        public DbSet<JobExclusion> JobExclusions { get; set; } = null!;

        /// <summary>
        /// Полный путь к файлу базы данных (.db).
        /// </summary>
        public string DbPath { get; }

        /// <summary>
        /// Конструктор контекста. Определяет местоположение файла БД.
        /// </summary>
        public DatabaseContext()
        {
            /// Используем стандартную папку LocalApplicationData (обычно
C:\Users\<User>\AppData\Local).
            /// Это гарантирует, что у приложения будут права на запись без прав
администратора.
            var folder = Environment.SpecialFolder.LocalApplicationData;
            var path = Environment.GetFolderPath(folder);

            /// Папка приложения: ...\\AppData\\Local\\BackupSync
            var appDataFolder = Path.Join(path, "BackupSync");

            /// Гарантируем, что папка существует
            Directory.CreateDirectory(appDataFolder);

            /// Итоговый путь: ...\\AppData\\Local\\BackupSync\\backupsync.db
            DbPath = Path.Join(appDataFolder, "backupsync.db");
        }
    }

```

```

    /// <summary>
    /// Настройка подключения к базе данных.
    /// </summary>
    protected override void OnConfiguring(DbContextOptionsBuilder options)
    {
        // Используем SQLite.
        // TODO [Future]: При необходимости можно добавить логирование SQL-
запросов:
        // options.LogTo(Console.WriteLine);
        options.UseSqlite($"Data Source={DbPath}");
    }

    /// <summary>
    /// Настройка схемы базы данных и связей между таблицами (Fluent API).
    /// </summary>
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // --- Связь: Device (1) <-> SyncJob (Many) ---
        modelBuilder.Entity<SyncJob>()
            .HasOne(j => j.Device)
            .WithMany(d => d.SyncJobs)
            .HasForeignKey(j => j.DeviceId)
            .OnDelete(DeleteBehavior.Cascade); // При удалении Устройства
удаляются все его Задания

        // --- Связь: SyncJob (1) <-> JobExclusion (Many) ---
        modelBuilder.Entity<JobExclusion>()
            .HasOne(e => e.SyncJob)
            .WithMany(j => j.Exclusions)
            .HasForeignKey(e => e.SyncJobId)
            .OnDelete(DeleteBehavior.Cascade); // При удалении Задания
удаляются все его Исключения
    }
}

```

```

    === BackupSync.Core\Services\DeviceService.cs ===
using System.Collections.Generic;
using System.Linq;
using System.Runtime.Versioning;
using System.Threading.Tasks;
using BackupSync.Core.Models;
using Microsoft.EntityFrameworkCore;

namespace BackupSync.Core.Services
{
    /// <summary>
    /// Сервис управления устройствами.

```

```

    /// Отвечает за сопоставление физически подключенных дисков (WMI) с
сохраненными в БД профилями.
    /// </summary>
    [SupportedOSPlatform("windows")]
    public class DeviceService
    {
        private readonly DatabaseContext _db;
        private readonly WmiService _wmi;

        public DeviceService(DatabaseContext dbContext, WmiService wmiService)
        {
            _db = dbContext;
            _wmi = wmiService;
        }

        /// <summary>
        /// Получает сводный статус всех устройств (подключенных и сохраненных).
        /// </summary>
        /// <returns>Список статусов для отображения в UI.</returns>
        public List<DeviceStatus> GetDeviceStatuses()
        {
            // 1. Получаем "живые" данные от WMI (Синхронно, безопасно для
STA/MTA)
            var connectedDisks = _wmi.GetLogicalDisks();

            // 2. Получаем сохраненные данные из БД
            var savedDevices = _db.Devices.ToList();

            var allStatuses = new List<DeviceStatus>();

            // Создаем словарь для быстрого поиска подключенных дисков по
SerialNumber
            var wmiLookup = connectedDisks.ToDictionary(d =>
d.VolumeSerialNumber!);

            // 3. Проходим по всем сохраненным устройствам
            foreach (var savedDevice in savedDevices)
            {
                // Пытаемся найти сохраненное устройство среди подключенных
                if (wmiLookup.TryGetValue(savedDevice.VolumeSerialNumber, out var
connectedDisk))
                {
                    // А. Устройство сохранено И подключено (Connected &
Recognized)

                    allStatuses.Add(new DeviceStatus
                    {
                        IsConnected = true,
                        IsRecognized = true,
                        DeviceId = savedDevice.DeviceId,
                        UserGivenName = savedDevice.UserGivenName,
                        WmiDisk = connectedDisk
                    });
                }
            }
        }
    }

```

```

    });

    // Удаляем из lookup, чтобы пометить как "обработанное"
    wmiLookup.Remove(savedDevice.VolumeSerialNumber);
}
else
{
    // Б. Устройство сохранено, но НЕ подключено (Disconnected &
Recognized)

    allStatuses.Add(new DeviceStatus
    {
        IsConnected = false,
        IsRecognized = true,
        DeviceId = savedDevice.DeviceId,
        UserGivenName = savedDevice.UserGivenName
        // WmiDisk = null
    });
}
}

// 4. Все, что осталось в wmiLookup – это новые, неизвестные
устройства
foreach (var connectedDisk in wmiLookup.Values)
{
    // В. Устройство подключено, но НЕ сохранено (Connected &
Unrecognized)

    allStatuses.Add(new DeviceStatus
    {
        IsConnected = true,
        IsRecognized = false,
        WmiDisk = connectedDisk,
        // Используем метку тома как временное имя
        UserGivenName = connectedDisk.VolumeLabel ?? "Новый том"
    });
}

    return allStatuses;
}

/// <summary>
/// Сохраняет новое физическое устройство в базу данных.
/// </summary>
public async Task<Device> AddNewDeviceAsync(LogicalDisk newDisk, string
userGivenName)
{
    var newDevice = new Device
    {
        UserGivenName = userGivenName,
        VolumeSerialNumber = newDisk.VolumeSerialNumber!,
        LastKnownLabel = newDisk.VolumeLabel,
        LastKnownLetter = newDisk.Letter
    }
}

```

```

    };
    _db.Devices.Add(newDevice);
    await _db.SaveChangesAsync();
    return newDevice;
}

/// <summary>
/// Перепривязывает существующий профиль устройства к новому физическому
носителю (или тому же после форматирования).
/// Обновляет SerialNumber, метку и букву.
/// </summary>
public async Task ReAssociateDeviceAsync(int lostDeviceId, LogicalDisk
newDisk)
{
    var deviceToUpdate = await _db.Devices.FindAsync(lostDeviceId);
    if (deviceToUpdate == null) return;

    deviceToUpdate.VolumeSerialNumber = newDisk.VolumeSerialNumber!;
    deviceToUpdate.LastKnownLabel = newDisk.Volumelabel;
    deviceToUpdate.LastKnownLetter = newDisk.Letter;

    _db.Devices.Update(deviceToUpdate);
    await _db.SaveChangesAsync();
}

/// <summary>
/// Удаляет устройство из базы данных.
/// Каскадно удаляет все связанные задания (SyncJobs).
/// </summary>
public async Task ForgetDeviceAsync(int deviceId)
{
    var deviceToForget = await _db.Devices
        .Include(d => d.SyncJobs)
        .FirstOrDefaultAsync(d => d.DeviceId == deviceId);

    if (deviceToForget == null) return;

    // Удаляем задания явно (хотя EF Core Cascade Delete может сделать
это сам, явное удаление безопаснее для логики)
    if (deviceToForget.SyncJobs.Any())
    {
        _db.SyncJobs.RemoveRange(deviceToForget.SyncJobs);
    }

    _db.Devices.Remove(deviceToForget);
    await _db.SaveChangesAsync();
}
}
}
}

```

```

        === BackupSync.Core\Services\JobService.cs ===
using System.Collections.Generic;
using System.Threading.Tasks;
using BackupSync.Core.Models;
using Microsoft.EntityFrameworkCore;

namespace BackupSync.Core.Services
{
    /// <summary>
    /// Сервис для управления заданиями синхронизации (CRUD).
    /// Отвечает за создание, чтение и удаление профилей бэкапа в базе данных.
    /// </summary>
    public class JobService
    {
        private readonly DatabaseContext _db;

        public JobService(DatabaseContext dbContext)
        {
            _db = dbContext;
        }

        /// <summary>
        /// Получает список всех сохраненных заданий.
        /// Загружает (Eager Loading) связанные данные об Устройстве для
        отображения в UI.
        /// </summary>
        public async Task<List<SyncJob>> GetJobsAsync()
        {
            return await _db.SyncJobs
                .Include(job => job.Device) // Подтягиваем имя устройства
                .ToListAsync();
        }

        /// <summary>
        /// Создает и сохраняет новое задание в БД.
        /// </summary>
        public async Task<SyncJob> CreateJobAsync(
            string jobName,
            string sourcePath,
            string destPath,
            int deviceId,
            JobType jobType,
            OrphanPolicy orphanPolicy,
            ConflictPolicy conflictPolicy)
        {
            var newJob = new SyncJob
            {
                JobName = jobName,
                SourcePath = sourcePath,
                DestPath = destPath,
                DeviceId = deviceId,
            }
        }
    }
}

```

```

        JobType = jobType,
        OrphanPolicy = orphanPolicy,
        ConflictPolicy = conflictPolicy
        // Exclusions по умолчанию пуст
    };

    _db.SyncJobs.Add(newJob);
    await _db.SaveChangesAsync();

    // Явно подгружаем связанное устройство, чтобы возвращаемый объект
    // содержал всю информацию для немедленного отображения в UI.
    await _db.Entry(newJob).Reference(j => j.Device).LoadAsync();

    return newJob;
}

/// <summary>
/// Удаляет задание по ID.
/// Связанные исключения (Exclusions) удаляются автоматически (Cascade
Delete в БД).
/// </summary>
public async Task DeleteJobAsync(int jobId)
{
    var jobToDelete = await _db.SyncJobs.FindAsync(jobId);

    if (jobToDelete == null) return;

    _db.SyncJobs.Remove(jobToDelete);
    await _db.SaveChangesAsync();
}

// TODO [Future]: Реализовать метод UpdateJobAsync для редактирования
существующих заданий.
}
}

```

```

=== BackupSync.Core\Services\LocalFileSystemProvider.cs ===
using System;
using System.Collections.Generic;
using System.IO;
using System.Threading.Tasks;
using BackupSync.Core.Abstractions;

namespace BackupSync.Core.Services
{
    /// <summary>
    /// Реализация провайдера файловой системы, использующая стандартные средства
    .NET (System.IO).
    /// Работает с локальными дисками и смонтированными сетевыми путями.
    /// </summary>

```

```

public class LocalFileSystemProvider : IFileSystemProvider
{
    /// <summary>
    /// Сканирует директорию и возвращает метаданные всех файлов.
    /// </summary>
    public Task<Dictionary<string, FileMetadata>> ScanDirectoryAsync(string
rootPath, List<string> exclusions)
    {
        var results = new Dictionary<string, FileMetadata>();

        if (!Directory.Exists(rootPath))
        {
            // Если исходной папки нет, возвращаем пустой результат (это не
ошибка, просто нечего копировать)
            return Task.FromResult(results);
        }

        // TODO [Future]: Directory.EnumerateFiles падает при ошибке доступа
к одной из подпапок (AccessDenied).
        // В будущем нужно заменить на кастомный рекурсивный обход с try-
catch.
        var allFiles = Directory.EnumerateFiles(rootPath, "*",
SearchOption.AllDirectories);

        foreach (var fullPath in allFiles)
        {
            // Получаем путь относительно корня (например,
"SubFolder\File.txt")
            string relativePath = Path.GetRelativePath(rootPath, fullPath);

            // TODO [Logic]: Реализовать проверку исключений (exclusions)
здесь

            // if (IsExcluded(relativePath, exclusions)) continue;

            var fileInfo = new FileInfo(fullPath);
            results[relativePath] = new FileMetadata
            {
                RelativePath = relativePath,
                Size = fileInfo.Length,
                LastWriteTimeUtc = fileInfo.LastWriteTimeUtc
            };
        }

        return Task.FromResult(results);
    }

    /// <summary>
    /// Копирует файл безопасно (через временный файл).
    /// </summary>
    public async Task CopyFileAsync(string sourceFullPath, string
destFullPath)

```



```

{
    // 1. Создаем папку назначения, если ее нет
    string? destDirectory = Path.GetDirectoryName(destFullPath);
    if (destDirectory != null)
    {
        Directory.CreateDirectory(destDirectory);
    }

    // 2. Используем временный файл для атомарности.
    // Если копирование прервется, у нас не останется "битого" файла с
    правильным именем.
    string tempFile = destFullPath + ".tmp";

    // Используем FileShare.Read, чтобы не блокировать файл, если его
    кто-то читает
    await using (var sourceStream = new FileStream(sourceFullPath,
        FileMode.Open, FileAccess.Read, FileShare.Read))
    await using (var tempStream = new FileStream(tempFile,
        FileMode.Create, FileAccess.Write, FileShare.None))
    {
        await sourceStream.CopyToAsync(tempStream);
    }

    // 3. Переименовываем временный файл в итоговый (с заменой старого)
    File.Move(tempFile, destFullPath, true);
}

/// <summary>
/// Удаляет файл. Идемпотентная операция (не падает, если файла уже нет).
/// </summary>
public Task DeleteFileAsync(string fullPath)
{
    if (File.Exists(fullPath))
    {
        File.Delete(fullPath);
    }
    return Task.CompletedTask;
}
}
}

```

```

=== BackupSync.Core\Services\SyncAnalyzerService.cs ===
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using BackupSync.Core.Abstractions;
using BackupSync.Core.Models;

namespace BackupSync.Core.Services
{

```

```

    /// <summary>
    /// Сервис, отвечающий за этап "Анализа".
    /// Сравнивает файловую структуру Источника и Назначения и выявляет различия
(Diffs).
    /// </summary>
    public class SyncAnalyzerService
    {
        private readonly IFileSystemProvider _fsProvider;

        public SyncAnalyzerService(IFileSystemProvider fsProvider)
        {
            _fsProvider = fsProvider;
        }

        /// <summary>
        /// Выполняет сравнение файлов в двух директориях.
        /// </summary>
        /// <param name="job">Объект задания (для получения списка исключений,
если нужно).</param>
        /// <param name="sourceRoot">Абсолютный путь к источнику.</param>
        /// <param name="destRoot">Абсолютный путь к назначению.</param>
        /// <returns>Список различий (<see cref="FileDiff"/>).</returns>
        public async Task<List<FileDiff>> AnalyzeAsync(SyncJob job, string
sourceRoot, string destRoot)
        {
            // TODO [Future]: Извлечь job.Exclusions и передать их в
ScanDirectoryAsync
            var exclusions = new List<string>();

            // Запускаем сканирование параллельно для ускорения
            var sourceTask = _fsProvider.ScanDirectoryAsync(sourceRoot,
exclusions);
            var destTask = _fsProvider.ScanDirectoryAsync(destRoot, exclusions);

            await Task.WhenAll(sourceTask, destTask);

            var sourceFiles = await sourceTask;
            var destFiles = await destTask; // Это изменяемый словарь (мы будем
удалять из него совпадения)

            var diffs = new List<FileDiff>();

            // 1. Проходим по файлам источника
            foreach (var sourcePair in sourceFiles)
            {
                var path = sourcePair.Key;
                var sourceMeta = sourcePair.Value;
                FileDiff diff;

                // Ищем такой же файл в назначении
                if (destFiles.TryGetValue(path, out var destMeta))

```

```

        {
            // Файл есть и там, и там -> Сравниваем
            diff = CompareMetadata(path, sourceMeta, destMeta);

            // Удаляем из списка файлов назначения, чтобы пометить как
            // обработанный
            destFiles.Remove(path);
        }
        else
        {
            // Файл есть только в источнике
            diff = new FileDiff
            {
                RelativePath = path,
                Status = FileStatus.OnlyInSource,
                SourceMetadata = sourceMeta
            };
        }
        diffs.Add(diff);
    }

    // 2. Все, что осталось в destFiles – это "сироты" (есть только в
    // Dest)
    foreach (var destPair in destFiles)
    {
        diffs.Add(new FileDiff
        {
            RelativePath = destPair.Key,
            Status = FileStatus.OnlyInDest,
            DestMetadata = destPair.Value
        });
    }

    return diffs;
}

/// <summary>
/// Сравнивает метаданные двух файлов и определяет их статус.
/// </summary>
private FileDiff CompareMetadata(string path, FileMetadata source,
FileMetadata dest)
{
    var status = FileStatus.Identical;

    // Разница во времени в секундах
    var timeDiff = (source.LastWriteTimeUtc -
dest.LastWriteTimeUtc).TotalSeconds;

    // FAT32 (флешки) имеет точность времени 2 секунды.
    // Если разница меньше 2 секунд, считаем время одинаковым.
    if (Math.Abs(timeDiff) > 2)

```

```

        {
            status = timeDiff > 0 ? FileStatus.SourceIsNewer :
FileStatus.DestIsNewer;
        }
        else if (source.Size != dest.Size)
        {
            // Если время "совпадает" (в пределах погрешности), но размер
разный – считаем Source новее (конфликт/изменение)
            // Можно было бы добавить статус "SizeMismatch", но для простоты
пусть будет SourceIsNewer
            status = FileStatus.SourceIsNewer;
        }

        return new FileDiff
        {
            RelativePath = path,
            Status = status,
            SourceMetadata = source,
            DestMetadata = dest
        };
    }
}
}

```

```

=== BackupSync.Core\Services\SyncExecutionService.cs ===
using System;
using System.Collections.Generic;
using System.IO;
using System.Threading.Tasks;
using BackupSync.Core.Abstractions;
using BackupSync.Core.Models;

namespace BackupSync.Core.Services
{
    /// <summary>
    /// Сервис, отвечающий за этап "Выполнения" (Execution).
    /// Принимает список действий (Actions) и физически применяет их к файловой
системе.
    /// </summary>
    public class SyncExecutionService
    {
        private readonly IFileSystemProvider _fsProvider;

        public SyncExecutionService(IFileSystemProvider fsProvider)
        {
            _fsProvider = fsProvider;
        }

        /// <summary>
        /// Выполняет список действий синхронизации.
    }
}

```



```

        Path.Combine(sourceRoot, path));
        break;

        // TODO: Реализовать DeleteOnSource, если мы решим
это поддерживать
    }
}
catch (Exception ex)
{
    // TODO [Log]: Здесь нужно добавить запись в лог ошибок (файл
или БД), чтобы пользователь знал о сбоях.
    System.Diagnostics.Debug.WriteLine($"ОШИБКА выполнения
{action.ActionType} для {path}: {ex.Message}");
}
}

return conflictsToResolve;
}
}
}
}

```

```

=== BackupSync.Core\Services\SyncPolicyService.cs ===
using System.Collections.Generic;
using System.Diagnostics;
using BackupSync.Core.Models;

namespace BackupSync.Core.Services
{
    /// <summary>
    /// Сервис, отвечающий за этап "Принятия решений" (Policy Engine).
    /// Принимает список различий (Diffs) и настройки задания (Job),
    /// и на их основе формирует список действий (Actions).
    /// </summary>
    public class SyncPolicyService
    {
        /// <summary>
        /// Применяет политики синхронизации к списку различий.
        /// </summary>
        /// <param name="diffs">Список найденных различий.</param>
        /// <param name="job">Задание с настройками политик (Orphan,
Conflict).</param>
        /// <returns>Список действий, которые нужно выполнить.</returns>
        public List<SyncAction> ApplyPolicies(List<FileDiff> diffs, SyncJob job)
        {
            var actions = new List<SyncAction>();

            Debug.WriteLine("--- Движок Политик: Начало ---");

            foreach (var diff in diffs)
            {

```

```

        var actionType = SyncActionType.Skip; // По умолчанию - ничего не
        делать

        // Логирование для отладки
        Debug.WriteLine($" -> Вход: {diff.Status} |
{diff.RelativePath}");

        switch (diff.Status)
        {
            case FileStatus.OnlyInSource:
                // Новый файл на источнике -> Копируем в назначение
                actionType = SyncActionType.CopyFromSourceToDest;
                Debug.WriteLine("        -> Решение:
CopyFromSourceToDest");
                break;

            case FileStatus.SourceIsNewer:
                // Файл изменился на источнике -> Обновляем назначение
                actionType = SyncActionType.CopyFromSourceToDest;
                Debug.WriteLine("        -> Решение: CopyFromSourceToDest
(Overwrite)");
                break;

            case FileStatus.Identical:
                // Файлы одинаковые -> Пропускаем
                actionType = SyncActionType.Skip;
                Debug.WriteLine("        -> Решение: Skip (Identical)");
                break;

            case FileStatus.OnlyInDest:
                // Файл есть только в назначении ("Сирота") -> Применяем
                политику сирот

                actionType = ApplyOrphanPolicy(job.OrphanPolicy);
                Debug.WriteLine($"        -> Решение (Сирота):
{actionType}");
                break;

            case FileStatus.DestIsNewer:
                // Файл в назначении новее ("Конфликт" для One-Way, норма
                для Two-Way) -> Применяем политику конфликтов
                actionType = ApplyConflictPolicy(job.ConflictPolicy);
                Debug.WriteLine($"        -> Решение (Конфликт):
{actionType}");
                break;

            default:
                Debug.WriteLine("        -> Решение: (UNKNOWN STATUS!)");
                break;
        }
    }

```

```

        actions.Add(new SyncAction { ActionType = actionType, Diff = diff
    });
    }

    Debug.WriteLine("--- Движок Политик: Конец ---");
    return actions;
}

/// <summary>
/// Определяет действие для "сирот" на основе выбранной политики.
/// </summary>
private SyncActionType ApplyOrphanPolicy(OrphanPolicy policy)
{
    return policy switch
    {
        OrphanPolicy.Ask => SyncActionType.AskUser,
        OrphanPolicy.KeepOnRemote => SyncActionType.Skip,
        OrphanPolicy.AutoDeleteOnRemote => SyncActionType.DeleteOnDest,
        // Для Two-Way режима: сирота на Dest = новый файл
        OrphanPolicy.AutoCopyToLocal =>
SyncActionType.CopyFromDestToSource,
        _ => SyncActionType.AskUser
    };
}

/// <summary>
/// Определяет действие для конфликтов (Dest is Newer) на основе
выбранной политики.
/// </summary>
private SyncActionType ApplyConflictPolicy(ConflictPolicy policy)
{
    return policy switch
    {
        ConflictPolicy.Ask => SyncActionType.AskUser,
        ConflictPolicy.KeepLocal => SyncActionType.CopyFromSourceToDest,
        // Перезаписать Dest старой версией с Source
        ConflictPolicy.KeepRemote => SyncActionType.CopyFromDestToSource,
        // Обновить Source новой версией с Dest
        ConflictPolicy.KeepNewest => SyncActionType.CopyFromDestToSource,
        // Dest и так новее
        _ => SyncActionType.AskUser
    };
}
}
}
}

```

=== BackupSync.Core\Services\WmiService.cs ===

```

using System;
using System.Collections.Generic;
using System.Linq;

```



```

using System.Management;
using System.Runtime.Versioning;
using System.Threading.Tasks;
using BackupSync.Core.Models;

namespace BackupSync.Core.Services
{
    /// <summary>
    /// Сервис для взаимодействия с Windows Management Instrumentation (WMI).
    /// Отвечает за обнаружение физических дисков и отслеживание событий
    подключения/отключения USB.
    /// </summary>
    [SupportedOSPlatform("windows")]
    public class WmiService : IDisposableable
    {
        private readonly ManagementEventWatcher _usbInsertWatcher;
        private readonly ManagementEventWatcher _usbRemoveWatcher;

        /// <summary>
        /// Событие, возникающее при изменении состава дисков (подключение или
        извлечение).
        /// </summary>
        public event Action? DisksChanged;

        /// <summary>
        /// Конструктор сервиса. Инициализирует и запускает мониторинг событий
        WMI.
        /// </summary>
        public WmiService()
        {
            // Подписка на событие ПОДКЛЮЧЕНИЯ (__InstanceCreationEvent) для
            LogicalDisk
            var insertQuery = new WqlEventQuery(
                "SELECT * FROM __InstanceCreationEvent WITHIN 2 " +
                "WHERE TargetInstance ISA 'Win32_LogicalDisk'");
            _usbInsertWatcher = new ManagementEventWatcher(insertQuery);
            _usbInsertWatcher.EventArrived += OnDisksChanged;
            _usbInsertWatcher.Start();

            // Подписка на событие ОТКЛЮЧЕНИЯ (__InstanceDeletionEvent) для
            LogicalDisk
            var removeQuery = new WqlEventQuery(
                "SELECT * FROM __InstanceDeletionEvent WITHIN 2 " +
                "WHERE TargetInstance ISA 'Win32_LogicalDisk'");
            _usbRemoveWatcher = new ManagementEventWatcher(removeQuery);
            _usbRemoveWatcher.EventArrived += OnDisksChanged;
            _usbRemoveWatcher.Start();
        }

        /// <summary>
        /// Обработчик событий WMI.
    }
}

```

```

    /// </summary>
    private void OnDisksChanged(object sender, EventArgs e)
    {
        // Используем небольшую задержку (Debounce), так как WMI может
        прислать
        // несколько событий подряд для одного физического подключения,
        // а системе нужно время, чтобы смонтировать букву диска.
        Task.Delay(500).ContinueWith(_ => { DisksChanged?.Invoke(); });
    }

    /// <summary>
    /// Получает список всех активных логических дисков в системе (только
    съемные и локальные).
    /// </summary>
    public List<LogicalDisk> GetLogicalDisks()
    {
        var disks = new List<LogicalDisk>();
        try
        {
            // DriveType=2 (Removable/USB), DriveType=3 (Local HDD/SSD)
            var query = new ManagementObjectSearcher(
                "SELECT * FROM Win32_LogicalDisk WHERE DriveType = 2 OR
                DriveType = 3");

            foreach (ManagementObject disk in
                query.Get().Cast<ManagementObject>())
            {
                var diskInfo = new LogicalDisk
                {
                    Letter = disk["DeviceID"]?.ToString() ?? string.Empty,
                    VolumeLabel = disk["VolumeName"]?.ToString(),
                    VolumeSerialNumber =
                        disk["VolumeSerialNumber"]?.ToString()
                };

                // Фильтруем пустые или системные разделы без серийника
                if (!string.IsNullOrEmpty(diskInfo.Letter) &&
                    !string.IsNullOrEmpty(diskInfo.VolumeSerialNumber))
                {
                    disks.Add(diskInfo);
                }
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine($"WMI Error (Sync): {ex.Message}");
        }
        return disks;
    }

    /// <summary>

```

```

    /// Освобождение ресурсов (остановка Watcher'ов).
    /// Вызывается DI-контейнером при завершении работы приложения.
    /// </summary>
    public void Dispose()
    {
        _usbInsertWatcher?.Stop();
        _usbInsertWatcher?.Dispose();

        _usbRemoveWatcher?.Stop();
        _usbRemoveWatcher?.Dispose();

        GC.SuppressFinalize(this);
    }
}
}

```

```

=== BackupSync.UI\Converters\InverseBoolToVisibilityConverter.cs ===
using System;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Data;

namespace BackupSync.UI.Converters
{
    /// <summary>
    /// Конвертер значения bool в Visibility с инверсией.
    /// Используется для скрытия элементов, когда условие истинно.
    /// <para>True -> Visibility.Collapsed (Скрыто)</para>
    /// <para>False -> Visibility.Visible (Видно)</para>
    /// </summary>
    public class InverseBoolToVisibilityConverter : IValueConverter
    {
        /// <summary>
        /// Преобразует bool в Visibility (Инвертировано).
        /// </summary>
        public object Convert(object value, Type targetType, object parameter,
string language)
        {
            // Безопасная проверка типа.
            // Если значение является bool и равно true -> Скрываем.
            if (value is bool boolValue && boolValue)
            {
                return Visibility.Collapsed;
            }

            // Во всех остальных случаях (false, null, другой тип) -> Показываем.
            return Visibility.Visible;
        }

        /// <summary>
        /// Обратное преобразование не поддерживается.
    }
}

```

```

        /// </summary>
        public object ConvertBack(object value, Type targetType, object
parameter, string language)
        {
            throw new NotImplementedException();
        }
    }
}

```

=== BackupSync.UI\ViewModels\DevicesViewModel.cs ===

```

using System;
using System.Collections.ObjectModel;
using System.Diagnostics;
using System.Linq;
using System.Threading.Tasks;
using BackupSync.Core.Models;
using BackupSync.Core.Services;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using Microsoft.UI.Dispatching;

namespace BackupSync.UI.ViewModels
{
    /// <summary>
    /// ViewModel для страницы управления устройствами.
    /// Отвечает за отображение списка дисков, добавление новых и удаление
старых.
    /// </summary>
    public partial class DevicesViewModel : ObservableObject
    {
        private readonly DeviceService _deviceService;
        private readonly WmiService _wmiService;

        // Очередь диспетчера для маршалинга вызовов в UI-поток
        private readonly DispatcherQueue _dispatcherQueue;

        /// <summary>
        /// Коллекция неопознанных (новых) подключенных дисков.
        /// </summary>
        public ObservableCollection<DeviceStatus> UnrecognizedDisks { get; } =
new();

        /// <summary>
        /// Основная коллекция устройств (все известные + подключенные).
        /// </summary>
        public ObservableCollection<DeviceStatus> Devices { get; } = new();

        // Ручная реализация свойства для избежания потенциальных AOT-
предупреждений
        private bool _hasUnrecognizedDisks;
    }
}

```

```

    /// <summary>
    /// Флаг: есть ли подключенные, но не добавленные диски (для отображения
плашки "Найден новый диск").
    /// </summary>
    public bool HasUnrecognizedDisks
    {
        get => _hasUnrecognizedDisks;
        set => SetProperty(ref _hasUnrecognizedDisks, value);
    }

    public DevicesViewModel(DeviceService deviceService, WmiService
wmiService)
    {
        _deviceService = deviceService;
        _wmiService = wmiService;

        // Получаем диспетчер текущего (UI) потока
        _dispatcherQueue = DispatcherQueue.GetForCurrentThread();

        // Подписываемся на события WMI (вставка/извлечение флешки)
        _wmiService.DisksChanged += OnDisksChanged;

        // Первичная загрузка
        LoadDevices();
    }

    /// <summary>
    /// Обработчик события изменения дисков от WMI.
    /// </summary>
    private void OnDisksChanged()
    {
        // WMI вызывает это событие из фонового потока.
        // Нам нужно обновить ObservableCollection, что можно делать ТОЛЬКО в
UI-потоке.
        _dispatcherQueue.TryEnqueue(() =>
        {
            Debug.WriteLine("WMI Event Received! Reloading devices...");
            LoadDevices();
        });
    }

    /// <summary>
    /// Синхронно перезагружает список устройств из сервиса.
    /// </summary>
    public void LoadDevices()
    {
        Devices.Clear();
        UnrecognizedDisks.Clear();

        // Получаем актуальный список статусов (WMI + DB)

```

```

        var statuses = _deviceService.GetDeviceStatuses();

        // Сортируем: сначала подключенные
        foreach (var status in statuses.OrderByDescending(d =>
d.IsConnected))
        {
            Devices.Add(status);

            // Если диск подключен, но не опознан — добавляем в список
            "Новых"

            if (status.IsConnected && !status.IsRecognized)
            {
                UnrecognizedDisks.Add(status);
            }
        }

        HasUnrecognizedDisks = UnrecognizedDisks.Any();
    }

    /// <summary>
    /// Асинхронно добавляет новое устройство в базу данных.
    /// </summary>
    public async Task AddDeviceAsync(DeviceStatus deviceToAdd, string
newName)
    {
        if (deviceToAdd == null || deviceToAdd.WmiDisk == null ||
string.IsNullOrEmpty(newName)) return;

        try
        {
            await _deviceService.AddNewDeviceAsync(deviceToAdd.WmiDisk,
newName);

            LoadDevices(); // Обновляем список, чтобы диск перешел в
катеорию "Сохраненные"
        }
        catch (Exception ex)
        {
            Debug.WriteLine($"Error adding device: {ex.Message}");
        }
    }

    /// <summary>
    /// Команда: Удалить ("Забыть") устройство из базы.
    /// </summary>
    [RelayCommand]
    private async Task ForgetDeviceAsync(DeviceStatus deviceToForget)
    {
        if (deviceToForget == null || !deviceToForget.DeviceId.HasValue)
return;

        try

```

```

        {
            await
_deviceService.ForgetDeviceAsync(deviceToForget.DeviceId.Value);
            LoadDevices(); // Обновляем список
        }
        catch (Exception ex)
        {
            Debug.WriteLine($"Error forgetting device: {ex.Message}");
        }
    }

    /// <summary>
    /// Асинхронно обновляет привязку "потерянного" устройства к новому
    физическому диску.
    /// </summary>
    public async Task ReAssociateDeviceAsync(DeviceStatus lostDevice,
DeviceStatus newDisk)
    {
        if (lostDevice == null || newDisk == null ||
!lostDevice.DeviceId.HasValue || newDisk.WmiDisk == null) return;

        try
        {
            await
_deviceService.ReAssociateDeviceAsync(lostDevice.DeviceId.Value,
newDisk.WmiDisk);
            LoadDevices(); // Обновляем список
        }
        catch (Exception ex)
        {
            Debug.WriteLine($"Error re-associating device: {ex.Message}");
        }
    }
}
}

```

=== BackupSync.UI\ViewModels\JobsViewModel.cs ===

```

using System;
using System.Collections.ObjectModel;
using System.Diagnostics;
using System.IO;
using System.Linq;
using System.Runtime.Versioning;
using System.Threading.Tasks;
using BackupSync.Core.Models;
using BackupSync.Core.Services;
using BackupSync.UI.Views.Dialogs;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using Microsoft.Extensions.DependencyInjection;

```

```

using Microsoft.UI.Xaml.Controls;

// Псевдонимы для разрешения конфликтов имен
using DeviceModel = BackupSync.Core.Models.Device;
using SystemPath = System.IO.Path;

namespace BackupSync.UI.ViewModels
{
    /// <summary>
    /// ViewModel для страницы управления заданиями (Jobs).
    /// Отвечает за создание, удаление и запуск синхронизации.
    /// </summary>
    public partial class JobsViewModel : ObservableObject
    {
        private readonly JobService _jobService;
        private readonly DeviceService _deviceService;
        private readonly SyncAnalyzerService _analyzerService;
        private readonly SyncPolicyService _policyService;
        private readonly SyncExecutionService _executionService;
        private readonly IServiceProvider _serviceProvider;

        /// <summary>
        /// Список всех сохраненных заданий.
        /// </summary>
        public ObservableCollection<SyncJob> Jobs { get; } = new();

        /// <summary>
        /// Список распознанных устройств (для выпадающего списка при создании
        задания).
        /// </summary>
        public ObservableCollection<DeviceModel> RecognizedDevices { get; } =
new();

        public JobsViewModel(
            JobService jobService,
            DeviceService deviceService,
            SyncAnalyzerService analyzerService,
            SyncPolicyService policyService,
            SyncExecutionService executionService,
            IServiceProvider serviceProvider)
        {
            _jobService = jobService;
            _deviceService = deviceService;
            _analyzerService = analyzerService;
            _policyService = policyService;
            _executionService = executionService;
            _serviceProvider = serviceProvider;

            // Запускаем асинхронную загрузку данных без блокировки конструктора
            (Fire-and-forget)
            _ = LoadDataAsync();
        }
    }
}

```



```

    }

    /// <summary>
    /// Асинхронно загружает список заданий и доступных устройств.
    /// </summary>
    public async Task LoadDataAsync()
    {
        Jobs.Clear();

        // Асинхронно получаем задания из БД
        var jobs = await _jobService.GetJobsAsync();
        foreach (var job in jobs)
        {
            Jobs.Add(job);
        }

        RecognizedDevices.Clear();

        // Получаем статусы устройств (синхронно, так как WMI/Reconciliation)
        var allDeviceStatuses = _deviceService.GetDeviceStatuses();

        // Фильтруем: нам нужны только те, что уже есть в базе (IsRecognized)
        и имеют DeviceId
        var recognized = allDeviceStatuses
            .Where(s => s.IsRecognized && s.DeviceId.HasValue)
            .Select(s => new DeviceModel
            {
                DeviceId = s.DeviceId!.Value,
                UserGivenName = s.UserGivenName!
            });

        foreach (var device in recognized)
        {
            RecognizedDevices.Add(device);
        }
    }

    /// <summary>
    /// Создает новое задание и сохраняет его в БД.
    /// </summary>
    public async Task CreateJobAsync(
        string jobName, string sourcePath, string destPath, int deviceId,
        JobType jobType, OrphanPolicy orphanPolicy, ConflictPolicy
conflictPolicy)
    {
        var newJob = await _jobService.CreateJobAsync(
            jobName, sourcePath, destPath, deviceId,
            jobType, orphanPolicy, conflictPolicy);

        Jobs.Add(newJob);
    }

```

```

    /// <summary>
    /// Удаляет выбранное задание.
    /// </summary>
    [RelayCommand]
    private async Task DeleteJobAsync(SyncJob? jobToDelete)
    {
        if (jobToDelete == null) return;

        await _jobService.DeleteJobAsync(jobToDelete.SyncJobId);
        Jobs.Remove(jobToDelete);
    }

    /// <summary>
    /// Запускает процесс синхронизации для выбранного задания.
    /// Состоит из 3 этапов: Анализ -> Политики -> Выполнение.
    /// </summary>
    [SupportedOSPlatform("windows")]
    [RelayCommand]
    private async Task SyncJobAsync(SyncJob? jobToSync)
    {
        if (jobToSync == null) return;

        Debug.WriteLine($"--- 🚀 НАЧАЛО СИНХРОНИЗАЦИИ: {jobToSync.JobName} -
--");

        // 1. Получаем свежую копию задания из БД (чтобы иметь актуальные
        // пути и политики)
        var job = (await _jobService.GetJobsAsync())
            .FirstOrDefault(j => j.SyncJobId == jobToSync.SyncJobId);

        if (job == null)
        {
            Debug.WriteLine($"ОШИБКА: Не удалось найти задание
{jobToSync.SyncJobId} в БД.");
            return;
        }

        // 2. Проверяем подключение устройства
        var deviceStatus = _deviceService.GetDeviceStatuses()
            .FirstOrDefault(d => d.DeviceId == job.DeviceId &&
d.IsConnected);

        if (deviceStatus == null)
        {
            Debug.WriteLine($"ОШИБКА: Устройство '{job.Device.UserGivenName}'
не подключено.");
            // TODO: Показать пользователю MessageBox
            return;
        }
    }

```

```

        // 3. Формируем пути
        string sourceRoot = job.SourcePath;
        // Убираем ведущие слэши, чтобы Combine сработал корректно
        string cleanDestPath = job.DestPath.TrimStart('\\', '/');
        string destRoot = SystemPath.Combine(deviceStatus.Letter,
cleanDestPath);

        try
        {
            // 4. ЭТАП 1: Анализ (Scan & Compare)
            var diffs = await _analyzerService.AnalyzeAsync(job, sourceRoot,
destRoot);

            Debug.WriteLine($"Анализ завершен. Найдено {diffs.Count}
расхождений.");

            // 5. ЭТАП 2: Применение политик (Decision Making)
            var actions = _policyService.ApplyPolicies(diffs, job);
            var actionsToExecute = actions.Where(a => a.ActionType !=
SyncActionType.Skip).ToList();

            // 6. ЭТАП 3: Выполнение "безопасных" действий
            Debug.WriteLine("Передача 'безопасных' команд 'Исполнителю'...");
            var unresolvedConflicts = await
_executionService.ExecuteAsync(actionsToExecute, sourceRoot, destRoot);
            Debug.WriteLine("'Исполнитель' завершил 'безопасные' действия.");

            // 7. Обработка конфликтов (если остались)
            if (unresolvedConflicts.Any())
            {
                Debug.WriteLine($"'Исполнитель' вернул
{unresolvedConflicts.Count} конфликтов (AskUser). Показ диалога...");

                var dialog = new
ConflictResolutionDialog(unresolvedConflicts);

                // Получаем MainWindow для привязки диалога (WinUI 3
требование)
                var mainWindow = _serviceProvider.GetService<MainWindow>();
                if (mainWindow == null) throw new NullReferenceException("Не
удалось получить MainWindow");

                dialog.XamlRoot = mainWindow.Content.XamlRoot;

                var result = await dialog.ShowAsync();

                if (result == ContentDialogResult.Primary)
                {
                    var resolvedActions = dialog.ResolvedActions;
                    Debug.WriteLine($"Пользователь решил
{resolvedActions.Count(a => a.ActionType != SyncActionType.Skip)} конфликтов.");

```

```

        await _executionService.ExecuteAsync(resolvedActions,
sourceRoot, destRoot);
        Debug.WriteLine("'Исполнитель' завершил 'решенные'
действия.");
    }
    else
    {
        Debug.WriteLine("Пользователь отменил разрешение
конфликтов.");
    }
}
else
{
    Debug.WriteLine("Все действия выполнены, конфликтов нет.");
}
}
catch (Exception ex)
{
    Debug.WriteLine($"ОШИБКА СИНХРОНИЗАЦИИ: {ex.Message}");
    // TODO: Показать пользователю MessageBox с ошибкой
}

    Debug.WriteLine($"--- ☒ ЗАВЕРШЕНИЕ СИНХРОНИЗАЦИИ: {job.JobName} ---
");
}
}
}
}

```

```

=== BackupSync.UI\Views\Dialogs\AddDeviceDialog.xaml.cs ===
using Microsoft.UI.Xaml.Controls;

namespace BackupSync.UI.Views.Dialogs
{
    /// <summary>
    /// Диалоговое окно для ввода имени нового устройства.
    /// </summary>
    public sealed partial class AddDeviceDialog : ContentDialog
    {
        /// <summary>
        /// Результат ввода пользователя (имя устройства).
        /// Доступно только после успешного закрытия диалога кнопкой "Сохранить".
        /// </summary>
        public string DeviceName { get; private set; } = string.Empty;

        /// <summary>
        /// Конструктор диалога.
        /// </summary>
        /// <param name="defaultName">Имя по умолчанию (обычно метка тома или
"Новый том").</param>
        public AddDeviceDialog(string defaultName)
    }
}

```

```

    {
        this.InitializeComponent();

        // Предзаполняем поле ввода
        NameTextBox.Text = defaultName;

        // Подписываемся на клик по кнопке "Primary" (Сохранить)
        this.PrimaryButtonClick += AddDeviceDialog_PrimaryButtonClick;
    }

    private void AddDeviceDialog_PrimaryButtonClick(ContentDialog sender,
ContentDialogButtonClickEventArgs args)
    {
        // Очищаем от лишних пробелов
        string inputName = NameTextBox.Text.Trim();

        // Простейшая валидация: имя не должно быть пустым
        if (string.IsNullOrEmpty(inputName))
        {
            // Отменяем закрытие диалога
            args.Cancel = true;

            // Визуально сообщаем об ошибке (меняем заголовок поля)
            NameTextBox.Header = "Название (обязательно)";
            // Можно также подсветить красным, но это требует стилей.
            // Изменения Header достаточно для MVP.
        }
        else
        {
            // Все ок, сохраняем результат
            DeviceName = inputName;
        }
    }
}
}
}

```

=== BackupSync.UI\Views\Dialogs\CreateJobDialog.xaml.cs ===

```

using System;
using System.Collections.ObjectModel;
using System.Linq;
using BackupSync.Core.Models;
using Microsoft.UI.Xaml.Controls;

// Псевдоним для удобства
using DeviceModel = BackupSync.Core.Models.Device;

namespace BackupSync.UI.Views.Dialogs
{
    /// <summary>
    /// Диалог создания нового задания синхронизации.

```

```

/// Собирает данные формы и возвращает их через публичные свойства.
/// </summary>
public sealed partial class CreateJobDialog : ContentDialog
{
    /// --- Публичные свойства для доступа к результатам ---

    public string JobName => JobNameTextBox.Text.Trim();
    public string SourcePath => SourcePathTextBox.Text.Trim();
    public string DestPath => DestPathTextBox.Text.Trim();

    /// <summary>
    /// ID выбранного устройства. Возвращает 0, если устройство не выбрано.
    /// </summary>
    public int SelectedDeviceId => (DeviceComboBox.SelectedItem as
DeviceModel)?.DeviceId ?? 0;

    /// <summary>
    /// Выбранный тип задания. Парсится из Tag выбранного ComboBoxItem.
    /// </summary>
    public JobType SelectedJobType =>
ParseEnumFromTag<JobType>(JobTypeComboBox);

    /// <summary>
    /// Выбранная политика "сирот".
    /// </summary>
    public OrphanPolicy SelectedOrphanPolicy =>
ParseEnumFromTag<OrphanPolicy>(OrphanPolicyComboBox);

    /// <summary>
    /// Выбранная политика конфликтов.
    /// </summary>
    public ConflictPolicy SelectedConflictPolicy =>
ParseEnumFromTag<ConflictPolicy>(ConflictPolicyComboBox);

    public CreateJobDialog(ObservableCollection<DeviceModel>
recognizedDevices)
    {
        this.InitializeComponent();

        // 1. Заполняем список устройств
        DeviceComboBox.ItemsSource = recognizedDevices;

        // Если есть устройства, выбираем первое по умолчанию
        if (recognizedDevices.Any())
        {
            DeviceComboBox.SelectedIndex = 0;
        }

        // 2. Подписываемся на клик по кнопке "Создать"
        this.PrimaryButtonClick += CreateJobDialog_PrimaryButtonClick;
    }
}

```

```

    }

    private void CreateJobDialog_PrimaryButtonClick(ContentDialog sender,
ContentDialogButtonClickEventArgs args)
    {
        bool hasError = false;

        // --- Валидация Названия ---
        if (string.IsNullOrEmpty(JobName))
        {
            JobNameTextBox.Header = "Название задания (Обязательно!)";
            hasError = true;
        }
        else
        {
            JobNameTextBox.Header = "Название задания"; // Сброс ошибки
        }

        // --- Валидация Источника ---
        if (string.IsNullOrEmpty(SourcePath))
        {
            SourcePathTextBox.Header = "Папка на компьютере (Обязательно!)";
            hasError = true;
        }
        else
        {
            SourcePathTextBox.Header = "Папка на компьютере (Источник)";
        }

        // --- Валидация Устройства ---
        if (SelectedDeviceId == 0)
        {
            DeviceComboBox.Header = "Целевое устройство (Не выбрано!)";
            hasError = true;
        }
        else
        {
            DeviceComboBox.Header = "Целевое устройство (USB)";
        }

        // --- Валидация Назначения (опционально, но желательно) ---
        if (string.IsNullOrEmpty(DestPath))
        {
            DestPathTextBox.Header = "Папка на устройстве (Обязательно!)";
            hasError = true;
        }
        else
        {
            DestPathTextBox.Header = "Папка на устройстве (Назначение)";
        }
    }

```

```

        // Если есть ошибки, предотвращаем закрытие диалога
        if (hasError)
        {
            args.Cancel = true;
        }
    }

    /// <summary>
    /// Вспомогательный метод для получения Enum из свойства Tag элемента
    ComboBox.
    /// </summary>
    private T ParseEnumFromTag<T>(ComboBox comboBox) where T : struct, Enum
    {
        if (comboBox.SelectedItem is ComboBoxItem item && item.Tag is string
tagString)
        {
            if (Enum.TryParse<T>(tagString, out var result))
            {
                return result;
            }
        }
        // Значение по умолчанию (первый элемент Enum), если парсинг не
удался
        return default;
    }
}
}
}

```

```

=== BackupSync.UI\Views\Dialogs\ReAssociateDialog.xaml.cs ===
using System.Collections.ObjectModel;
using BackupSync.Core.Models;
using Microsoft.UI.Xaml.Controls;

namespace BackupSync.UI.Views.Dialogs
{
    /// <summary>
    /// Диалог выбора нового физического диска для существующего (но
отключенного) профиля устройства.
    /// </summary>
    public sealed partial class ReAssociateDialog : ContentDialog
    {
        /// <summary>
        /// Выбранный пользователем новый диск.
        /// Свойство будет заполнено только при успешном нажатии "Сохранить".
        /// </summary>
        public DeviceStatus? SelectedDevice { get; private set; }

        /// <summary>
        /// Конструктор диалога.
        /// </summary>
    }
}

```



```

        /// <param name="lostDevice">Устройство из базы данных, которое потеряло
связь (Source).</param>
        /// <param name="unrecognizedDisks">Список доступных новых дисков
(Candidates).</param>
        public ReAssociateDialog(DeviceStatus lostDevice,
ObservableCollection<DeviceStatus> unrecognizedDisks)
        {
            this.InitializeComponent();

            // Формируем понятное сообщение для пользователя
            InfoTextBlock.Text = $"Устройство '{lostDevice.DisplayName}'
сохранено в базе, но не найдено. " +
                $"Выберите новый физический диск, чтобы связать
этот профиль с ним.";

            // Привязываем список кандидатов
            DisksComboBox.ItemsSource = unrecognizedDisks;

            // Выбираем первый элемент по умолчанию для удобства
            if (unrecognizedDisks.Count > 0)
            {
                DisksComboBox.SelectedIndex = 0;
            }

            this.PrimaryButtonClick += OnPrimaryButtonClick;
        }

        private void OnPrimaryButtonClick(ContentDialog sender,
ContentDialogButtonClickEventArgs args)
        {
            // Проверяем, выбран ли диск
            if (DisksComboBox.SelectedItem is DeviceStatus selected)
            {
                SelectedDevice = selected;
            }
            else
            {
                // Если ничего не выбрано, запрещаем закрытие диалога
                args.Cancel = true;
                DisksComboBox.Header = "Выберите диск (Обязательно!)";
            }
        }
    }
}

```

```

=== BackupSync.UI\Views\ConflictResolutionDialog.xaml.cs ===
using System;
using System.Collections.Generic;
using System.Linq;
using BackupSync.Core.Models;

```

```

using Microsoft.UI.Xaml.Controls;

namespace BackupSync.UI.Views.Dialogs
{
    /// <summary>
    /// Диалог для ручного разрешения конфликтов синхронизации.
    /// Позволяет пользователю выбрать действие для каждого проблемного файла.
    /// </summary>
    public sealed partial class ConflictResolutionDialog : ContentDialog
    {
        // Исходный список конфликтов (только для чтения)
        private readonly List<SyncAction> _conflicts;

        /// <summary>
        /// Список принятых решений. Изначально заполнен действиями "Skip".
        /// Обновляется по мере выбора пользователем вариантов в ComboBox.
        /// </summary>
        public List<SyncAction> ResolvedActions { get; private set; } = new();

        public ConflictResolutionDialog(List<SyncAction> conflicts)
        {
            this.InitializeComponent();
            _conflicts = conflicts;

            // 1. Привязываем данные к UI
            ConflictsListView.ItemsSource = _conflicts;

            // 2. Инициализируем список решений значениями по умолчанию
            foreach (var conflict in _conflicts)
            {
                // Создаем новый объект Action, чтобы не портить исходный,
                // пока пользователь не подтвердит выбор.
                // По умолчанию ставим Skip (безопасное действие).
                ResolvedActions.Add(new SyncAction
                {
                    ActionType = SyncActionType.Skip,
                    Diff = conflict.Diff
                });
            }
        }

        /// <summary>
        /// Обработчик изменения выбора в выпадающем списке строки.
        /// </summary>
        private void ResolutionComboBox_SelectionChanged(object sender,
        SelectionChangedEventArgs e)
        {
            // 1. Получаем ComboBox и связанный с ним объект SyncAction (через
            Tag)

            if (sender is not ComboBox comboBox) return;
            if (comboBox.Tag is not SyncAction originalAction) return;

```

```

        // 2. Находим соответствующее действие в нашем списке результатов
        var targetAction = ResolvedActions.FirstOrDefault(a =>
a.Diff.RelativePath == originalAction.Diff.RelativePath);

        if (targetAction == null) return;

        // 3. Получаем выбранный тип действия из Tag элемента ComboBoxItem
        // (Это надежнее, чем switch по SelectedIndex)
        if (comboBox.SelectedItem is ComboBoxItem selectedItem &&
selectedItem.Tag is string actionString)
        {
            if (Enum.TryParse<SyncActionType>(actionString, out var newType))
            {
                targetAction.ActionType = newType;
            }
        }
    }
}
}
}

```

=== BackupSync.UI\Views\DevicesPage.xaml.cs ===

```

using System;
using BackupSync.Core.Models;
using BackupSync.UI.ViewModels;
using BackupSync.UI.Views.Dialogs;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;

namespace BackupSync.UI.Views
{
    /// <summary>
    /// Страница управления устройствами.
    /// Отвечает за отображение списка и обработку UI-событий (открытие
    диалогов).
    /// </summary>
    public sealed partial class DevicesPage : Page
    {
        /// <summary>
        /// ViewModel страницы. Свойство доступно для x:Bind в XAML.
        /// </summary>
        public DevicesViewModel ViewModel { get; }

        public DevicesPage()
        {
            this.InitializeComponent();

            // Получаем экземпляр ViewModel из DI-контейнера
            ViewModel = App.Current.Services.GetService<DevicesViewModel>()

```

```
        ?? throw new InvalidOperationException("DevicesViewModel  
не зарегистрирована в DI.");
```

```
        // Устанавливаем DataContext для стандартных Binding'ов  
        this.DataContext = ViewModel;  
    }
```

```
    /// <summary>  
    /// Обработчик кнопки "Сохранить" (Добавить новое устройство).  
    /// </summary>
```

```
private async void AddDeviceButton_Click(object sender, RoutedEventArgs  
e)
```

```
{  
    // Извлекаем объект DeviceStatus из свойства Tag кнопки  
    if ((sender as Button)?.Tag is not DeviceStatus deviceToAdd)  
        return;
```

```
    // Создаем диалог  
    var dialog = new AddDeviceDialog(deviceToAdd.DisplayName);
```

```
    // [ВАЖНО] XamlRoot обязателен для WinUI 3  
    dialog.XamlRoot = this.XamlRoot;
```

```
    var result = await dialog.ShowAsync();
```

```
    if (result == ContentDialogResult.Primary &&  
!string.IsNullOrEmpty(dialog.DeviceName))  
    {  
        // Передаем данные во ViewModel  
        await ViewModel.AddDeviceAsync(deviceToAdd, dialog.DeviceName);  
    }  
}
```

```
    /// <summary>  
    /// Обработчик кнопки "Найти диск..." (Перепривязать потерянное  
устройство).
```

```
    /// </summary>
```

```
private async void ReAssociateButton_Click(object sender, RoutedEventArgs  
e)
```

```
{  
    // Извлекаем "потерянное" устройство из Tag  
    if ((sender as Button)?.Tag is not DeviceStatus lostDevice)  
        return;
```

```
    // Создаем диалог, передавая список кандидатов (неопознанных дисков)  
    var dialog = new ReAssociateDialog(lostDevice,  
ViewModel.UnrecognizedDisks);
```

```
    dialog.XamlRoot = this.XamlRoot;
```

```
    var result = await dialog.ShowAsync();
```

```

        if (result == ContentDialogResult.Primary && dialog.SelectedDevice !=
null)
        {
            // Передаем данные во ViewModel
            await ViewModel.ReAssociateDeviceAsync(lostDevice,
dialog.SelectedDevice);
        }
    }
}
}

```

=== BackupSync.UI\Views\JobsPage.xaml.cs ===

```

using System;
using BackupSync.UI.ViewModels;
using BackupSync.UI.Views.Dialogs;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;

namespace BackupSync.UI.Views
{
    /// <summary>
    /// Страница управления заданиями (Jobs).
    /// </summary>
    public sealed partial class JobsPage : Page
    {
        /// <summary>
        /// ViewModel страницы.
        /// </summary>
        public JobsViewModel ViewModel { get; }

        public JobsPage()
        {
            this.InitializeComponent();

            // Получаем ViewModel из DI-контейнера
            ViewModel = App.Current.Services.GetService<JobsViewModel>()
                ?? throw new InvalidOperationException("JobsViewModel не
зарегистрирована в DI.");

            // Устанавливаем DataContext
            this.DataContext = ViewModel;
        }

        /// <summary>
        /// Обработчик кнопки "Создать новое задание".
        /// Открывает диалоговое окно мастера создания задания.
        /// </summary>
    }
}

```

```

        private async void CreateJobButton_Click(object sender, RoutedEventArgs
e)
        {
            // 1. Создаем диалог, передаем список известных устройств из
ViewModel
            var dialog = new CreateJobDialog(ViewModel.RecognizedDevices);

            // 2. Указываем родительское окно (обязательно для WinUI 3)
            dialog.XamlRoot = this.XamlRoot;

            // 3. Показываем диалог
            var result = await dialog.ShowAsync();

            // 4. Если пользователь нажал "Создать" (и валидация внутри диалога
прошла успешно)
            if (result == ContentDialogResult.Primary)
            {
                // 5. Передаем данные из диалога во ViewModel для создания
задания

                await ViewModel.CreateJobAsync(
                    dialog.JobName,
                    dialog.SourcePath,
                    dialog.DestPath,
                    dialog.SelectedDeviceId,
                    dialog.SelectedJobType,
                    dialog.SelectedOrphanPolicy,
                    dialog.SelectedConflictPolicy
                );
            }
        }
    }
}

```

=== BackupSync.UI\App.xaml.cs ===

```

using System;
using BackupSync.Core.Abstractions;
using BackupSync.Core.Services;
using BackupSync.UI.ViewModels;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml;

namespace BackupSync.UI
{
    /// <summary>
    /// Точка входа в приложение WinUI 3.
    /// Отвечает за инициализацию DI-контейнера и создание главного окна.
    /// </summary>
    public partial class App : Application
    {

```

```

        // Ссылка на главное окно (необходима, чтобы сборщик мусора не удалил
окно)
        private Window? m_window;

        /// <summary>
        /// Глобальный провайдер сервисов (DI Container).
        /// </summary>
        public IServiceProvider Services { get; }

        /// <summary>
        /// Типизированный доступ к текущему экземпляру приложения.
        /// </summary>
        public static new App Current => (App)Application.Current;

        public App()
        {
            this.InitializeComponent();
            Services = ConfigureServices();
        }

        /// <summary>
        /// Настройка внедрения зависимостей (Dependency Injection).
        /// Здесь регистрируются все сервисы, ViewModels и окна.
        /// </summary>
        private static IServiceProvider ConfigureServices()
        {
            var services = new ServiceCollection();

            // --- 1. Data Layer (Данные и БД) ---
            services.AddDbContext<DatabaseContext>();

            // --- 2. Infrastructure (Системные сервисы) ---
            // WmiService должен быть Singleton, чтобы держать активную подписку
на события USB.
            services.AddSingleton<WmiService>();

            // Абстракция файловой системы (используем стандартную System.IO)
            services.AddTransient<IFileSystemService,
LocalFileSystemService>();

            // --- 3. Domain Logic (Бизнес-логика) ---
            services.AddTransient<DeviceService>();
            services.AddTransient<JobService>();
            services.AddTransient<SyncAnalyzerService>();
            services.AddTransient<SyncPolicyService>();
            services.AddTransient<SyncExecutionService>();

            // --- 4. UI Layer (ViewModels & Windows) ---
            // DevicesViewModel делаем Singleton, чтобы список устройств не
перезагружался при переключении вкладок.
            services.AddSingleton<DevicesViewModel>();

```

```

        // JobsViewModel создается заново при необходимости (Transient).
        services.AddTransient<JobsViewModel>();

        // Главное окно - Singleton (в нашем приложении только одно окно).
        services.AddSingleton<MainWindow>();

        return services.BuildServiceProvider();
    }

    /// <summary>
    /// Вызывается при запуске приложения конечным пользователем.
    /// </summary>
    protected override void OnLaunched(LaunchActivatedEventArgs args)
    {
        // Получаем главное окно из DI-контейнера
        m_window = Services.GetService<MainWindow>();

        if (m_window == null)
        {
            throw new InvalidOperationException("Не удалось разрешить
зависимость MainWindow из DI-контейнера.");
        }

        m_window.Activate();
    }
}
}

```

=== BackupSync.UI\MainWindow.xaml.cs ===

```

using System;
using BackupSync.UI.Views;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;

namespace BackupSync.UI
{
    /// <summary>
    /// Логика главного окна (Code-behind).
    /// Отвечает за навигацию между страницами (DevicesPage, JobsPage).
    /// </summary>
    public sealed partial class MainWindow : Window
    {
        public MainWindow()
        {
            this.InitializeComponent();

            // Заголовок окна задается в XAML или здесь.
            // События (Loaded, SelectionChanged) уже привязаны в XAML,
            // поэтому здесь их дублировать не нужно.
        }
    }
}

```



```

    }

    /// <summary>
    /// Вызывается, когда NavigationView полностью загружен.
    /// Используется для установки начальной страницы.
    /// </summary>
    private void AppNavigationView_Loaded(object sender, RoutedEventArgs e)
    {
        // Выбираем первый пункт меню ("Устройства") программно при старте.
        // Это автоматически вызовет событие SelectionChanged и загрузит
        страницу.
        if (AppNavigationView.MenuItems.Count > 0)
        {
            AppNavigationView.SelectedItem = AppNavigationView.MenuItems[0];
        }
    }

    /// <summary>
    /// Вызывается при смене выбранного пункта меню.
    /// </summary>
    private void AppNavigationView_SelectionChanged(NavigationView sender,
        NavigationViewSelectionChangedEventArgs args)
    {
        // 1. Проверяем, выбраны ли Настройки (нижняя кнопка)
        if (args.IsSettingsSelected)
        {
            // TODO: Создать SettingsPage и раскомментировать
            // ContentFrame.Navigate(typeof(SettingsPage));
        }
        // 2. Проверяем обычные пункты меню
        else if (args.SelectedItemContainer != null)
        {
            // Получаем Tag из XAML ("devices" или "jobs")
            var tag = args.SelectedItemContainer.Tag?.ToString();

            Type pageType = tag switch
            {
                "devices" => typeof(DevicesPage),
                "jobs" => typeof(JobsPage),
                _ => typeof(DevicesPage) // Fallback
            };

            // Выполняем навигацию, только если мы еще не на этой странице
            if (ContentFrame.CurrentSourcePageType != pageType)
            {
                ContentFrame.Navigate(pageType, null,
                    args.RecommendedNavigationTransitionInfo);
            }
        }
    }
}

```

```
}
```

```
=== BackupSync.UI\Views\ConflictResolutionDialog.xaml ===
<ContentDialog
  x:Class="BackupSync.UI.Views.Dialogs.ConflictResolutionDialog"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  xmlns:models="using:BackupSync.Core.Models"
  mc:Ignorable="d"
  Title="Решение конфликтов"
  PrimaryButtonText="ОК"
  CloseButtonText="Отмена"
  DefaultButton="Primary"
  FullSizeDesired="True">

  <Grid>
    <Grid.RowDefinitions>
      <RowDefinition Height="Auto"/>
      <RowDefinition Height="Auto"/>
      <RowDefinition Height="*/>
    </Grid.RowDefinitions>

    <TextBlock Grid.Row="0"
      Text="При выполнении операции резервного копирования  
встретились файлы с одинаковыми именами. Выберите  
действие:"
      TextWrapping="Wrap"
      Margin="0,0,0,16"/>

    <Grid Grid.Row="1" Padding="12,0,24,8" Opacity="0.6">
      <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*/>
        <ColumnDefinition Width="220"/>
      </Grid.ColumnDefinitions>
      <TextBlock Grid.Column="0" Text="Сохранить оба файла"
        Style="{ThemeResource CaptionTextBlockStyle}"/>
      <TextBlock Grid.Column="1" Text="Позже"
        Style="{ThemeResource CaptionTextBlockStyle}"/>
    </Grid>

    <ListView Grid.Row="2" x:Name="ConflictsListView" SelectionMode="None">
      <ListView.ItemTemplate>
        <DataTemplate x:DataType="models:SyncAction">
          <Grid HorizontalAlignment="Stretch" Padding="0,8">
            <Grid.ColumnDefinitions>
              <ColumnDefinition Width="*/>
              <ColumnDefinition Width="220"/>
            </Grid.ColumnDefinitions>
```

```

        <StackPanel Grid.Column="0" VerticalAlignment="Center"
Margin="0,0,16,0">
            <TextBlock Text="{Binding Diff.RelativePath}"
                FontWeight="SemiBold"
                TextTrimming="CharacterEllipsis"
                TooltipService.ToolTip="{Binding
Diff.RelativePath}"/>

            <StackPanel Orientation="Horizontal" Spacing="4"
Opacity="0.7">
                <TextBlock Text="РұЦЪРёC†РёPSP°:"/>
                <TextBlock Text="{Binding Diff.Status}"/>
            </StackPanel>
        </StackPanel>

        <ComboBox Grid.Column="1"
            x:Name="ResolutionComboBox"
            Tag="{Binding}"
            HorizontalAlignment="Stretch"
            SelectedIndex="0"
            SelectionChanged="ResolutionComboBox_SelectionC
hanged">

            <ComboBoxItem Content="РұЦЪРёSPiCfCfC,РёC,Сђ"
Tag="Skip"/>

            <ComboBoxItem Content="РұРµЦЪРµР·Р°РiРёCfP°C,Сђ USB
(Keep Local)" Tag="CopyFromSourceToDest"/>
            <ComboBoxItem Content="РұРµЦЪРµР·Р°РiРёCfP°C,Сђ РұРё
(Keep Remote)" Tag="CopyFromDestToSource"/>

        </ComboBox>

    </Grid>
</DataTemplate>
</ListView.ItemTemplate>
</ListView>

</Grid>
</ContentDialog>

```

=== BackupSync.UI\Views\DevicesPage.xaml ===

```

<Page
    x:Name="MyDevicesPage"
    x:Class="BackupSync.UI.Views.DevicesPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:models="using:BackupSync.Core.Models"

```

```

mc:Ignorable="d"
Background="{ThemeResource ApplicationPageBackgroundThemeBrush}"

<Grid Margin="24">
    <StackPanel Spacing="16">

        <TextBlock Text="PJPiCßP°PIP»PµPSPëPµ CíCíC, CßPSPN°CíC, PIP°PjPë"
Style="{ThemeResource TitleLargeTextBlockStyle}"/>
        <TextBlock Text="P–PrPµCíCß PsC, PSp±CßP°P¶P°CßC, CíCµ PICíPµ
CíPSc...CßP°PSPµPSPSC<Pµ Pë PïPSPrPeP»CßC‡PµPSPSC<Pµ CíPµPN°C‡P°Cí USB-
PSP°PePSPiPëC, PµP»Pë."
Style="{ThemeResource BodyTextBlockStyle}" Opacity="0.6"/>

        <ListView ItemsSource="{Binding Devices}" SelectionMode="None">
            <ListView.ItemTemplate>
                <DataTemplate x:DataType="models:DeviceStatus">
                    <Grid Padding="0,12" HorizontalAlignment="Stretch"
BorderBrush="{ThemeResource SystemControlForegroundBaseLowBrush}"
BorderThickness="0,0,0,1">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="Auto"/>
                            <ColumnDefinition Width="*/>
                            <ColumnDefinition Width="Auto"/>
                            <ColumnDefinition Width="Auto"/>
                        </Grid.ColumnDefinitions>

                        <FontIcon Grid.Column="0" Glyph="⌵"
FontSize="24" VerticalAlignment="Center" Margin="0,0,16,0"/>

                        <StackPanel Grid.Column="1"
VerticalAlignment="Center">
                            <TextBlock Text="{Binding DisplayName}"
Style="{ThemeResource SubtitleTextBlockStyle}"/>
                            <StackPanel Orientation="Horizontal" Spacing="4"
Opacity="0.6">
                                <TextBlock Text="P‘CíPePIP°:"/>
                                <TextBlock Text="{Binding Letter}"
FontWeight="SemiBold"/>
                                <TextBlock Text="|"/>
                                <TextBlock Text="S/N:"/>
                                <TextBlock Text="{Binding
VolumeSerialNumber}"/>
                            </StackPanel>
                        </StackPanel>

                        <StackPanel Grid.Column="2"
VerticalAlignment="Center" Margin="24,0,24,0" Orientation="Horizontal"
Spacing="8">
                            <StackPanel Orientation="Horizontal" Spacing="6"
Visibility="{x:Bind IsConnected, Mode=OneWay}">

```

```

        <FontIcon Glyph="&#xE930;" FontSize="12"
Foreground="{ThemeResource SystemFillColorSuccessBrush}"/>
        <TextBlock Text="PµPsPrPeP»CfC†PµPS"
Foreground="{ThemeResource SystemFillColorSuccessBrush}"/>
        </StackPanel>

        <StackPanel Orientation="Horizontal" Spacing="6"
Visibility="{x:Bind IsConnected, Mode=OneWay, Converter={StaticResource
InverseBoolToVisibilityConverter}}">
        <FontIcon Glyph="&#xE9CE;" FontSize="12"
Opacity="0.5"/>
        <TextBlock Text="PhC,PeP»CfC†PµPS"
Opacity="0.5"/>
        </StackPanel>
    </StackPanel>

    <StackPanel Grid.Column="3" Orientation="Horizontal"
Spacing="8">

        <Button Content="PŸPsC...CfP°PSPëC,Ch"
Style="{ThemeResource AccentButtonStyle}"
Visibility="{x:Bind IsRecognized,
Mode=OneWay, Converter={StaticResource InverseBoolToVisibilityConverter}}"
Click="AddDeviceButton_Click"
Tag="{Binding}"/>

        <Button Content="PĶP°PN°C,Pë PrPëCfPe..."
Visibility="{x:Bind CanBeReAssociated,
Mode=OneWay}"
IsEnabled="{Binding
DataContext.HasUnrecognizedDisks, ElementName=MyDevicesPage}"
Click="ReAssociateButton_Click"
Tag="{Binding}"
ToolTipService.ToolTip="PŸPICŲP·P°C,Ch
CĶC,PsC, PïCfPSc,,PëP»Ch Cf PSPsPICŲPj C,,PëP·PëC†PµCfPePëPj PrPëCfPePSPj"/>

        <Button Content="PJPrP°P»PëC,Ch"
Visibility="{x:Bind IsRecognized,
Mode=OneWay}"
Command="{Binding
DataContext.ForgetDeviceCommand, ElementName=MyDevicesPage}"
CommandParameter="{Binding}">
        <Button.Resources>
            <ResourceDictionary>
                <SolidColorBrush
x:Key="ButtonBackgroundPointerOver" Color="#33FF0000"/>
                <SolidColorBrush
x:Key="ButtonForegroundPointerOver" Color="Red"/>
                <SolidColorBrush
x:Key="ButtonBorderBrushPointerOver" Color="Red"/>
            </ResourceDictionary>

```

```

        </Button.Resources>
    </Button>

    </StackPanel>
</Grid>
</DataTemplate>
</ListView.ItemTemplate>
</ListView>

</StackPanel>
</Grid>
</Page>

```

```

=== BackupSync.UI\Views\Dialogs\AddDeviceDialog.xaml ===
<ContentDialog
    x:Class="BackupSync.UI.Views.Dialogs.AddDeviceDialog"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d"
    Title="P”PSP°PIPëC, Çb CfCfC, ÇbPSPM°CfC, PIPs"
    PrimaryButtonText="PŸPSC...ÇbP°PSPëC, Çb"
    CloseButtonText="PñC, PjPµPSP°"
    DefaultButton="Primary">

    <StackPanel Spacing="16" MinWidth="350">

        <TextBlock Text="PµÇbPëPrCfPjP°PÑC, Pµ PïPSPSCÇC, PSPSPµ PëPjCÇ P”P»CÇ
        ÇKÇ, PsPiPs CfCfC, ÇbPSPM°CfC, PIP°, C‡C, PsP±C< P»PµPiPePs CfP·PSP°PIP°C, Çb PµPiPs PI
        P±CfPrCfC%PµPj:"
            TextWrapping="Wrap"
            Style="{ThemeResource BodyTextBlockStyle}"/>

        <TextBox x:Name="NameTextBox"
            Header="PkP°P·PIP°PSPëPµ"
            PlaceholderText="PKP°PïÇbPëPjPµÇb: P P°P±PsC‡P°CÇ
            C,,P»PµC€PeP°"/>

    </StackPanel>
</ContentDialog>

```

```

=== BackupSync.UI\Views\Dialogs\CreateJobDialog.xaml ===
<ContentDialog
    x:Class="BackupSync.UI.Views.Dialogs.CreateJobDialog"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"

```

[illegible]

```

PlaceholderText="РќР°РІСЂРјРјСЂ: \Backups\MyProjects"
Description="РћС,РSPСЃРјС,РμР»СђРSC<Р№ РІСЃС,Сђ РSC,
РєРSCЂРSCЃ С,,Р»РμСєРєРј (Р±РμР· Р±СЃРјРІС< РјРјСЃРјР°)" />

<TextBlock Text="РμРSP»РјС,РјРјРј СЂР°Р·СЂμСєРμРSPРјСЃ
РєРSPSC,,Р»РјРјС,РSPИ" Style="{ThemeResource SubtitleTextBlockStyle}"
Margin="0,12,0,0"/>

<ComboBox x:Name="OrphanPolicyComboBox"
Header="Р·СЃР»Рј С,,Р°Р№Р»» РμСЃС,Сђ С,РSP»СђРєРSP РSP° USB
(РјРјСЂСЂС,Р°)"
HorizontalAlignment="Stretch"
SelectedIndex="0">
<ComboBoxItem Content="РќРμ С,СђРSPИР°С,Сђ (РћСЃС,Р°РІРјС,Сђ
С,,Р°Р№Р»»)" Tag="KeepOnRemote"/>
<ComboBoxItem Content="РјРІСЂР°СєРјРІР°С,Сђ
РІРSP»СђР·РSPИР°С,РμР»СЃ" Tag="Ask"/>
<ComboBoxItem Content="РјРјР°Р»Р»РјС,Сђ СЃ USB (Р—РμСЂРјР°Р»Р»РSP)"
Tag="AutoDeleteOnRemote"/>
<ComboBoxItem Content="РљРSPИРјСРSPИР°С,Сђ РSP° РμРљ
(Р’РSPСЃСЃС,Р°РSPSPИР»РμРSPРμ)" Tag="AutoCopyToLocal"/>
</ComboBox>

<ComboBox x:Name="ConflictPolicyComboBox"
Header="Р·СЃР»Р»Рј С,,Р°Р№Р»» РјР·РјРSPμРSP СЃ РSP±РμРјС...
СЃС,РSCЂРSP (РљРSPSC,,Р»РјРјС,)"
HorizontalAlignment="Stretch"
SelectedIndex="0">
<ComboBoxItem Content="РјРІСЂР°СєРјРІР°С,Сђ
РІРSP»СђР·РSPИР°С,РμР»СЃ" Tag="Ask"/>
<ComboBoxItem Content="РћСЃС,Р°РІР»СЃС,Сђ РІРμСЂСЃРјС СЃ РμРљ"
Tag="KeepLocal"/>
<ComboBoxItem Content="РћСЃС,Р°РІР»СЃС,Сђ РІРμСЂСЃРјС СЃ USB"
Tag="KeepRemote"/>
<ComboBoxItem Content="РћСЃС,Р°РІР»СЃС,Сђ СЃР°РјСЃРј РSPSPИСЃС
(РІРSP РР°С,Рμ)" Tag="KeepNewest"/>
</ComboBox>

</StackPanel>
</ScrollViewer>
</ContentDialog>

```

```

=== BackupSync.UI\Views\Dialogs\ReAssociateDialog.xaml ===
<ContentDialog
x:Class="BackupSync.UI.Views.Dialogs.ReAssociateDialog"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:models="using:BackupSync.Core.Models"

```



```

mc:Ignorable="d"
Title="РұРµСЃРµРїСЃРёРІСЃРµРµ° СЃСЃСЃ,СЃРµР№СЃСЃ,РІР°°"
PrimaryButtonText="РёРµС...СЃР°°РРёС,Сё СЃРІСЃРµР·Сё"
CloseButtonText="РёС,РјРµРРР°°"
DefaultButton="Primary">

<StackPanel Spacing="16" MinWidth="400">

    <TextBlock x:Name="InfoTextBlock"
        TextWrapping="Wrap"
        Style="{ThemeResource BodyTextBlockStyle}"/>

    <ComboBox x:Name="DisksComboBox"
        Header="Р'С<Р±РµСЃРёС,Рµ РРРРІС<Р№ С,,РёР·РёС±РµСЃРёРёР№
Prёёёёё"
        PlaceholderText="РёРїРёёёёРё РРРµРРРРРРР·РРР°РРРРРРР<С...
PrёёёёёРРРРРРР..."
        HorizontalAlignment="Stretch">

        <ComboBox.ItemTemplate>
            <DataTemplate x:DataType="models:DeviceStatus">
                <TextBlock>
                    <Run Text="{Binding DisplayName}" FontWeight="SemiBold"/>
                    <Run Text=" ("/>
                    <Run Text="{Binding Letter}"/>
                    <Run Text=", S/N: "/>
                    <Run Text="{Binding VolumeSerialNumber}"/>
                    <Run Text=")"/>
                </TextBlock>
            </DataTemplate>
        </ComboBox.ItemTemplate>
    </ComboBox>

</StackPanel>
</ContentDialog>

```

=== BackupSync.UI\Views\JobsPage.xaml ===

```

<Page
    x:Name="MyJobsPage"
    x:Class="BackupSync.UI.Views.JobsPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:models="using:BackupSync.Core.Models"
    mc:Ignorable="d"
    Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">

    <Grid Margin="24">
        <Grid.RowDefinitions>

```

```

        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*/>
    </Grid.RowDefinitions>

    <StackPanel Grid.Row="0" Spacing="4" Margin="0,0,0,12">
        <TextBlock Text="P-P°PrP°PSPëCŮ ĆŦPëPSC...ĆŦPsPSPëP·P°C†PëPë"
Style="{ThemeResource TitleLargeTextBlockStyle}"/>
        <TextBlock Text="PĶP°CŦĆ,ĆŦPsPN°C,Pμ, PëP°PëPëPμ PïP°PïPëPë Pë
PëĆŦPrP° PSCŦPŲPSPs PëPsPïPëĆŦPsPIP°C,ĆĤ."
Style="{ThemeResource BodyTextBlockStyle}" Opacity="0.6"/>
    </StackPanel>

    <StackPanel Grid.Row="1" Orientation="Horizontal" Margin="0,0,0,16">
        <Button x:Name="CreateJobButton"
Click="CreateJobButton_Click"
Style="{ThemeResource AccentButtonStyle}">
            <StackPanel Orientation="Horizontal" Spacing="8">
                <FontIcon Glyph="⚙️"/>
                <TextBlock Text="PŸPsP·PrP°C,ĆĤ PSPsPIPsPμ P·P°PrP°PSPëPμ"/>
            </StackPanel>
        </Button>
    </StackPanel>

    <ListView Grid.Row="2"
ItemsSource="{Binding Jobs}"
SelectionMode="None">

        <ListView.ItemTemplate>
            <DataTemplate x:DataType="models:SyncJob">
                <Grid HorizontalAlignment="Stretch" Padding="0,12"
BorderBrush="{ThemeResource SystemControlForegroundBaseLowBrush}"
BorderThickness="0,0,0,1">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="Auto"/>
                        <ColumnDefinition Width="*/>
                        <ColumnDefinition Width="Auto"/>
                    </Grid.ColumnDefinitions>

                    <FontIcon Grid.Column="0" Glyph="⚙️" FontSize="24"
VerticalAlignment="Center" Margin="0,0,16,0"/>

                    <StackPanel Grid.Column="1" VerticalAlignment="Center"
Margin="0,0,16,0">
                        <TextBlock Text="{Binding JobName}"
Style="{ThemeResource SubtitleTextBlockStyle}"/>

                        <StackPanel Orientation="Horizontal" Spacing="4"
Opacity="0.7">
                            <TextBlock Text="{Binding SourcePath}"
FontWeight="SemiBold" TextTrimming="CharacterEllipsis" MaxWidth="250"/>

```

```

                                <FontIcon Glyph="⌂" FontSize="10"
VerticalAlignment="Center"/>
                                <TextBlock Text="{Binding Device.UserGivenName}"
FontWeight="SemiBold"/>
                                <TextBlock Text="\"/>
                                <TextBlock Text="{Binding DestPath}"
TextTrimming="CharacterEllipsis"/>
                                </StackPanel>
                                </StackPanel>

                                <StackPanel Grid.Column="2" Orientation="Horizontal"
Spacing="8">

                                    <Button ToolTipService.ToolTip="⌂"
CíPëPSC...CßPsPSPëP·P°C†PëCñ"
                                    Command="{Binding DataContext.SyncJobCommand,
ElementName=MyJobsPage}"
                                    CommandParameter="{Binding}"
                                    <StackPanel Orientation="Horizontal" Spacing="8">
                                        <FontIcon Glyph="⌂"/>
Foreground="{ThemeResource SystemFillColorSuccessBrush}"/>
                                        <TextBlock Text="PŸC,P°CßC,"/>
                                        </StackPanel>
                                    </Button>

                                    <Button ToolTipService.ToolTip="PJPŕP°P»PëC,Cß
P·P°PrP°PSPëPµ"
                                    Command="{Binding
DataContext.DeleteJobCommand, ElementName=MyJobsPage}"
                                    CommandParameter="{Binding}"
                                    <FontIcon Glyph="⌂"/>
                                    <Button.Resources>
                                        <ResourceDictionary>
                                            <SolidColorBrush
x:Key="ButtonForegroundPointerOver" Color="Red"/>
                                        </ResourceDictionary>
                                    </Button.Resources>
                                    </Button>
                                </StackPanel>
                            </Grid>
                        </DataTemplate>
                    </ListView.ItemTemplate>
                </ListView>
            </Grid>
        </Page>

```

=== BackupSync.UI\App.xaml ===

```

<Application
    x:Class="BackupSync.UI.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

```

```

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="using:BackupSync.UI"
xmlns:converters="using:BackupSync.UI.Converters">

<Application.Resources>
    <ResourceDictionary>
        <ResourceDictionary.MergedDictionaries>
            <XamlControlsResources xmlns="using:Microsoft.UI.Xaml.Controls"
/>

        </ResourceDictionary.MergedDictionaries>

        <converters:InverseBoolToVisibilityConverter
x:Key="InverseBoolToVisibilityConverter"/>

    </ResourceDictionary>
</Application.Resources>
</Application>

```

=== BackupSync.UI\MainWindow.xaml ===

```

<Window
    x:Class="BackupSync.UI.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d"
    Title="BackupSync">

    <Window.SystemBackdrop>
        <MicaBackdrop Kind="BaseAlt"/>
    </Window.SystemBackdrop>

    <NavigationView x:Name="AppNavigationView"
        PaneDisplayMode="LeftCompact"
        IsSettingsVisible="True"
        IsBackButtonVisible="Collapsed"
        Loaded="AppNavigationView_Loaded"
        SelectionChanged="AppNavigationView_SelectionChanged">

        <NavigationView.MenuItems>
            <NavigationViewItem Content="ПЈСЃЃ,СЃРѕРЃЃЃ,РІР°" Tag="devices">
                <NavigationViewItem.Icon>
                    <FontIcon Glyph="⦿"/>
                </NavigationViewItem.Icon>
            </NavigationViewItem>

            <NavigationViewItem Content="Р–Р°РРР°РРР°РР" Tag="jobs">
                <NavigationViewItem.Icon>
                    <FontIcon Glyph="⦿"/>
                </NavigationViewItem.Icon>
            </NavigationViewItem>
        </NavigationView.MenuItems>
    </NavigationView>

```

```
        </NavigationViewItem>
    </NavigationView.MenuItems>

    <Frame x:Name="ContentFrame" Padding="24">
        <Frame.ContentTransitions>
            <TransitionCollection>
                <NavigationThemeTransition/>
            </TransitionCollection>
        </Frame.ContentTransitions>
    </Frame>

</NavigationView>
</Window>
```

Скриншот работы приложения

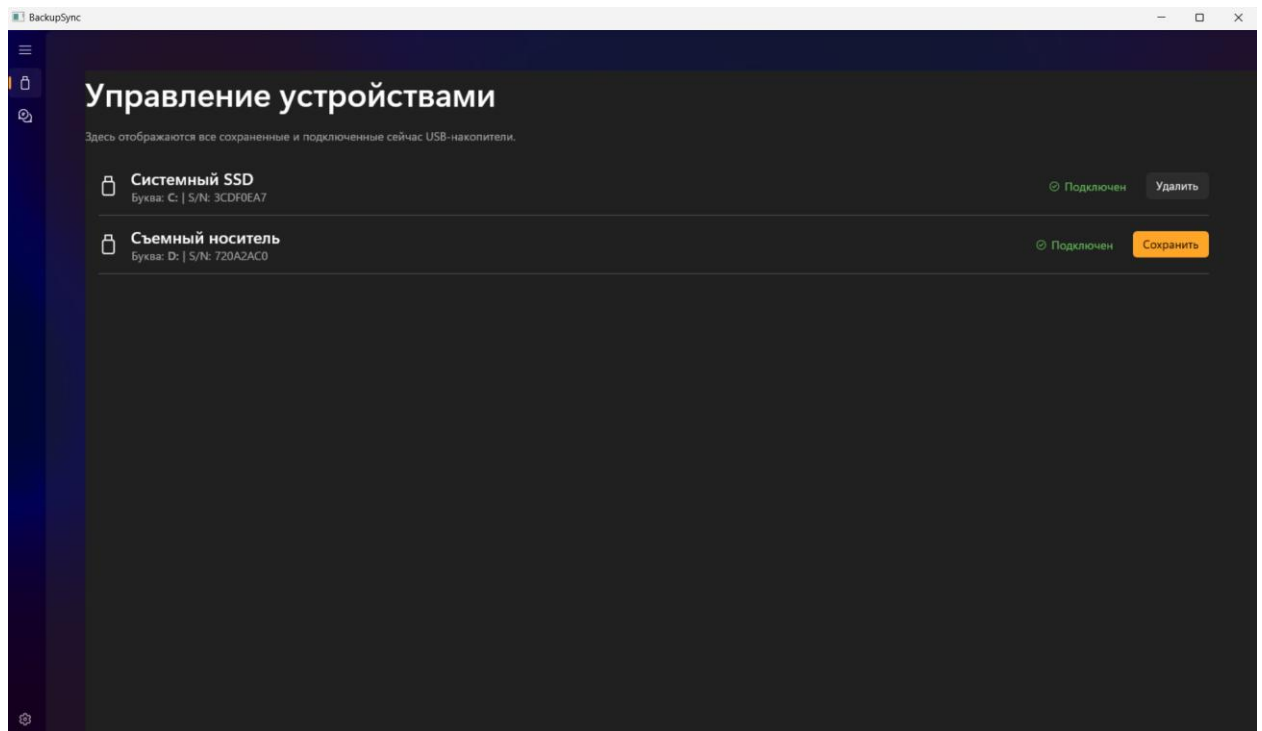


Рисунок 1. Основное меню

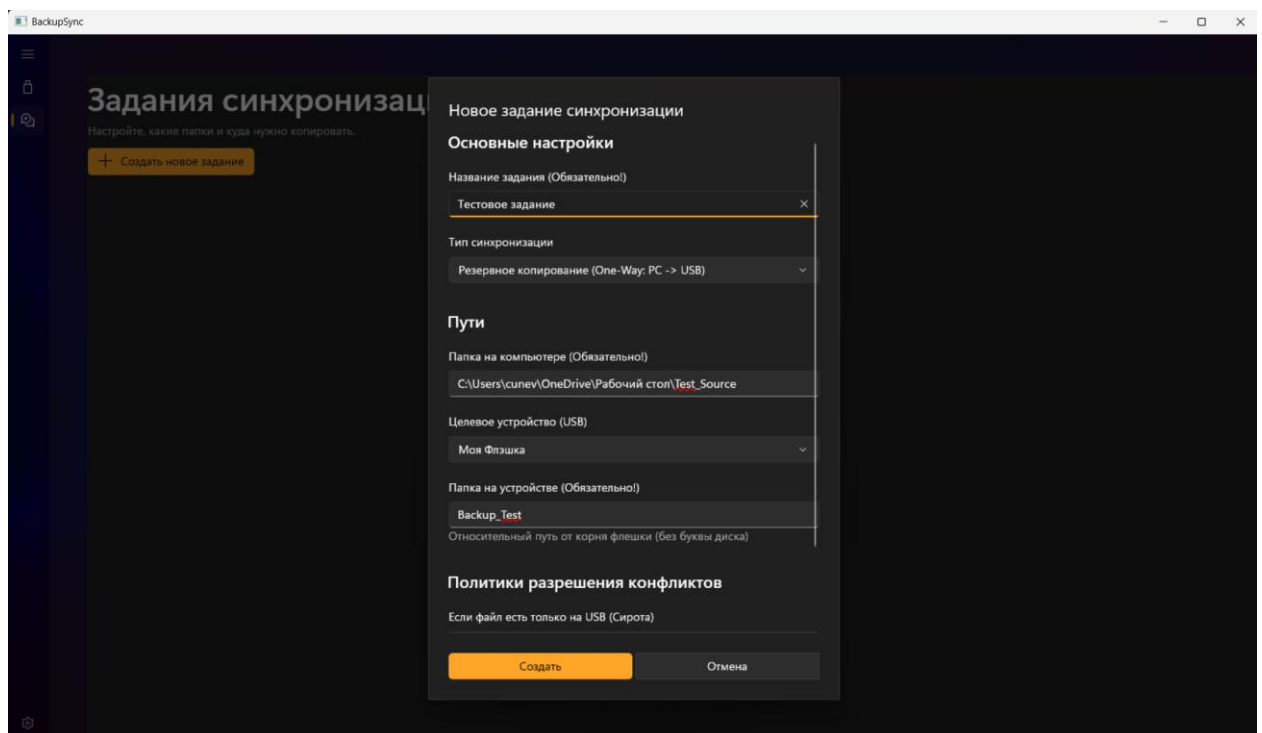


Рисунок 2. Создание задания синхронизации

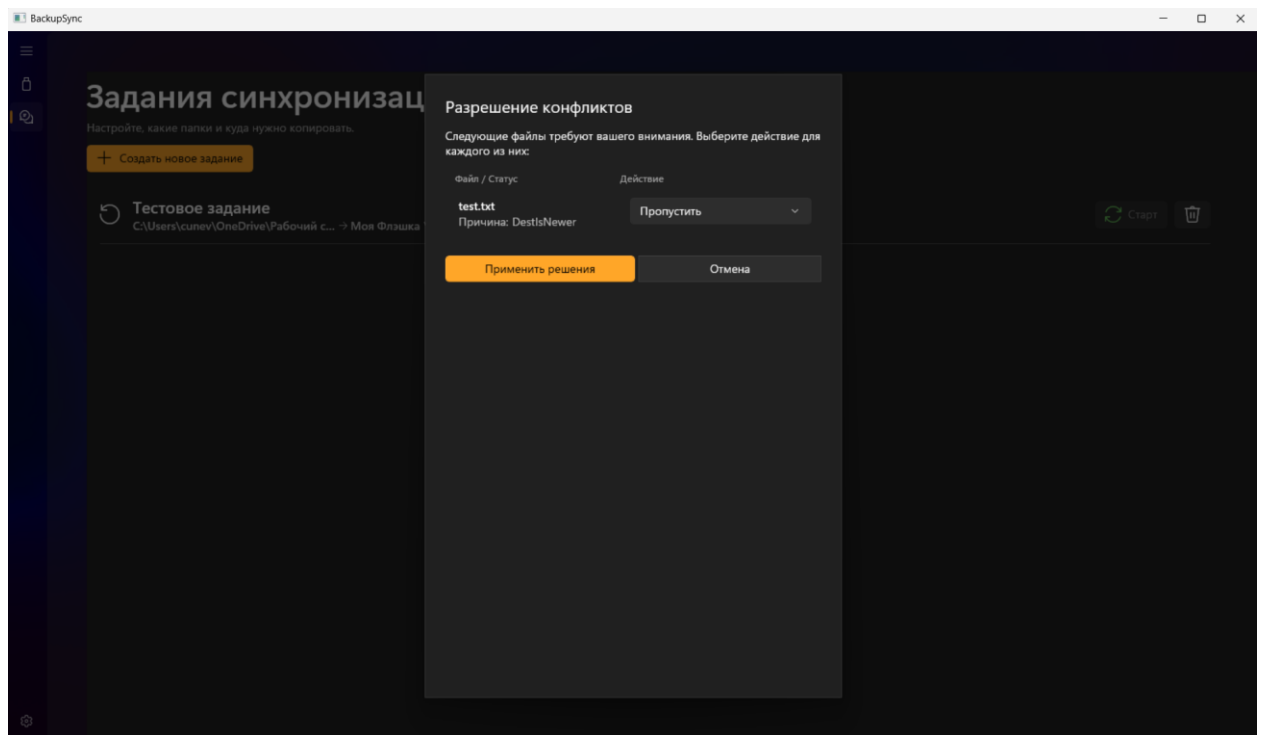


Рисунок 3. Решение конфликтов

Заключение

В ходе выполнения проекта был освоен переход от динамически типизированного Python к статически типизированному C#. Изучены механизмы Dependency Injection (внедрение зависимостей) для управления временем жизни сервисов и ViewModels в WinUI-приложении. Реализованный проект «BackupSync» успешно решает задачу аппаратной привязки профилей синхронизации к конкретным серийным номерам томов (VSN).